Groovy HTTP Client User Guide

Will Woodman

Version 2.1.0, 2026-10-26

Table of Contents

Getting Started . . . 1
1. Introduction . . . 1
1.1. Built on a modern platform . . . 1
1.2. Public HTTP/3 test endpoints . . . 1
1.3. Requirements . . . 2
2. Core Features and Capabilities . . . 2
2.1. What it deliberately does not do . . . 3
3. History . . . 3
4. Quick Start . . . 4
4.1. Step 1: Check your tools . . . 4
4.2. Step 2: Get the library . . . 4
4.3. Step 3: Make your first request . . . 5
4.4. Step 4: Add query parameters and headers . . . 5
4.5. Step 5: Read JSON into your own class . . . 5
4.6. Step 6: Handle an error . . . 5
4.7. Step 7: Send data . . . 6
4.8. Step 8: Go asynchronous . . . 6
4.9. Step 9: Configure the client for real use . . . 6
4.10. Step 10: Test it without the network . . . 6
4.11. Before you go to production . . . 7
5. How the Client Works . . . 7
5.1. The core classes . . . 7
5.2. The request pipeline . . . 9
5.3. The DSL . . . 9
5.4. Threads . . . 11
Common Usage . . . 12
6. Public Test Sites . . . 12
7. The Test Platform . . . 12
7.1. Example 1: stub a service and check what your code sent . . . 13
7.2. Example 2: the same with the test DSL . . . 14
7.3. Example 3: a Spock specification with the trait . . . 14
7.4. Example 4: test the unhappy paths . . . 14
7.5. Example 5: test asynchronous code . . . 15
7.6. Choosing between public sites and the test platform . . . 17
8. Creating a Client . . . 17
8.1. Choosing how to create it . . . 17
8.2. The constructor . . . 17
8.3. The builder . . . 18
8.4. Profiles and externalised settings . . . 18
8.5. Calling more than one host . . . 19
8.6. Default headers . . . 19
8.7. Lifecycle . . . 19
9. GET Requests . . . 20
9.1. Paths . . . 20

9.2. Query parameters . . . 20
9.3. Headers . . . 21
9.4. Reading the response . . . 21
9.5. Typed results . . . 21
9.6. Conditional GET . . . 22
9.7. HEAD and OPTIONS . . . 22
10. REST API Patterns . . . 22
10.1. Pagination . . . 22
10.2. Filtering and sorting . . . 24
10.3. API versions . . . 24
11. Sending Data . . . 25
11.1. The verbs and what they mean . . . 25
11.2. Choosing how to send the body . . . 25
11.3. JSON . . . 26
11.4. Forms . . . 26
11.5. Other content types . . . 26
11.6. Multipart documents . . . 26
11.7. Redirects after a POST . . . 27
11.8. Making POST safe to retry . . . 27
12. Cookies . . . 28
12.1. How the client handles cookies . . . 28
12.2. A session login . . . 28
12.3. Setting and reading cookies yourself . . . 29
12.4. Cookie policy . . . 29
12.5. Cookies and security . . . 30
12.6. Testing code that uses cookies . . . 30
13. Handling Results . . . 30
13.1. Exceptions . . . 30
13.2. Response handlers . . . 31
13.3. Typed results . . . 31
13.4. Problem details . . . 31
13.5. Which to choose . . . 32
14. Sync, Async and Streaming . . . 32
14.1. Synchronous: the default choice . . . 32
14.2. Asynchronous: for concurrency and composition . . . 33
14.3. Streaming: for big or endless bodies . . . 33
14.4. Decision guide . . . 34
15. Security . . . 34
15.1. What the client protects you from . . . 34
15.2. Setting up security in development . . . 35
15.3. Settings you will change . . . 37
Configuration . . . 38
16. The Configuration Model . . . 38
17. Client Configuration . . . 38
17.1. Base URL and allowed hosts . . . 39
17.2. Connection and timeouts . . . 39

17.3. Protocol . . . 40
17.4. TLS . . . 40
17.5. Proxy . . . 41
17.6. Redirects . . . 41
17.7. Cookies . . . 41
17.8. Content: codecs and compression . . . 41
17.9. Authentication . . . 42
17.10. Default headers and response handlers . . . 42
17.11. Resilience: retries and the circuit breaker . . . 42
17.12. Limits . . . 43
17.13. Observability: logging and metrics . . . 43
17.14. Profiles . . . 44
18. Codecs and Data Transformation . . . 44
18.1. Where codecs are used . . . 44
18.2. The built-in codecs . . . 45
18.3. Media type matching . . . 45
18.4. Typed results: mapping onto your classes . . . 46
18.5. Jackson . . . 46
18.6. Your own codecs . . . 47
18.7. Transforming data in the request and response . . . 47
19. Request Configuration . . . 48
19.1. URI and query . . . 48
19.2. Headers . . . 48
19.3. Cookies . . . 48
19.4. Content . . . 48
19.5. Multipart . . . 49
19.6. Authentication . . . 49
19.7. Compression . . . 49
19.8. Timeouts . . . 49
19.9. Retry . . . 49
19.10. Protocol . . . 50
19.11. Response handling . . . 50
19.12. Streaming targets . . . 50
20. Response Configuration . . . 50
20.1. Status handlers . . . 50
20.2. Exception handler . . . 51
20.3. Parsers and typed results . . . 51
20.4. Headers . . . 51
20.5. Decoded content . . . 51
20.6. Size limits . . . 51
20.7. Stream consumers . . . 52
21. Verb Forms . . . 52
21.1. Anatomy of a call . . . 52
21.2. The forms . . . 52
21.3. Typed forms in detail . . . 53
21.4. Streaming verbs . . . 53

21.5. Multipart verbs . . . 54
21.6. Writing the calls in Groovy . . . 54
21.7. Exceptions . . . 54
22. Error Handling . . . 55
22.1. HttpResponseException . . . 55
22.2. Problem details . . . 55
22.3. ResponseTooLargeException . . . 55
22.4. All exceptions . . . 56
Advanced Features . . . 57
23. Authentication . . . 57
23.1. The schemes . . . 57
23.2. Where credentials go . . . 57
23.3. OAuth2 client credentials . . . 57
24. Proxies . . . 58
24.1. Proxy authentication . . . 58
24.2. SOCKS and HTTP/3 . . . 58
25. Streaming in Depth . . . 59
25.1. Downloads . . . 59
25.2. Lines, Server-Sent Events and NDJSON . . . 59
25.3. Stopping early . . . 60
25.4. Limits and timeouts for long streams . . . 60
25.5. Uploads . . . 60
26. Compression . . . 60
26.1. Compressing requests . . . 61
27. Retries, Circuit Breaker and Metrics . . . 61
27.1. Retries . . . 61
27.2. The circuit breaker . . . 62
27.3. Inspecting the breaker . . . 63
27.4. Reacting to state changes . . . 63
27.5. Collecting metrics . . . 63
28. HTTP/3 in Depth . . . 64
28.1. Discovery modes . . . 65
28.2. Fallback . . . 65
29. Interceptors . . . 65
29.1. Per-attempt timings with Micrometer . . . 65
30. URL Builder . . . 66
Logging and Testing . . . 67
31. Logging . . . 67
32. Testing Your Code . . . 67
32.1. Adding the fixtures . . . 68
32.2. Asynchronous tests . . . 68
32.3. MockHttpServer . . . 68
32.4. HttpClientTestDsl . . . 69
32.5. HttpClientTestTrait . . . 69
Reference . . . 71
33. Examples . . . 71

33.1. Synchronous . . . 71
33.2. Asynchronous . . . 72
33.3. Streaming . . . 73
34. Upgrading . . . 74
34.1. From 2.0 to 2.1 . . . 74
34.2. From 1.x to 2.0 . . . 75
35. Troubleshooting . . . 75
36. API Map and Licence . . . 78
36.1. Classes you use . . . 78
36.2. Getting help . . . 79
36.3. Licence . . . 79

Getting Started

Chapter 1. Introduction

The Groovy HTTP Client is a Groovy DSL for making HTTP requests, built on the JDK's own `java.net.http.HttpClient`. You describe a request in a closure and get back a response, a parsed value, or one of your own types:

```
def client = new GroovyHttpClient('https://httpbin.org')

def r = client.getSync('/get') { query(lang: 'groovy') }
assert r.statusCode == 200
assert r.json.args.lang == 'groovy'
```

It is usable from Groovy and Java, including statically compiled (`@CompileStatic`) Groovy.

1.1. Built on a modern platform

Groovy 6 and Java 25. The library needs Java 25 or later and Groovy 6. It is compiled to Java 25 bytecode and tested on JDK 25 and JDK 27.

HTTP/3 with graceful fallback. On JDK 26 and later the client can use HTTP/3 (QUIC). Where HTTP/3 is not available — an older JDK, a server that does not offer it, a network that blocks UDP — it falls back to HTTP/2, which in turn falls back to HTTP/1.1. You opt in with one call, and the same code runs everywhere:

```
def client = GroovyHttpClient.builder('https://quic.nginx.org').http3().build()
println client.getSync('/').httpVersion     // HTTP_3 on JDK 26+, HTTP_2 otherwise
```

Virtual threads. Every request runs on a virtual thread, so thousands of concurrent requests need no thread pool tuning, and a blocking call costs almost nothing.

1.2. Public HTTP/3 test endpoints

Endpoints maintained to serve HTTP/3, for trying the client, your tools or your proxies. The notes record what this client saw with JDK 27 in October 2026, using the default discovery mode:

Endpoint	Implementation	Seen with this client
quic.nginx.org	nginx's own HTTP/3 test site	HTTP/3 from the first request
cloudflare-quic.com	Cloudflare's quiche; shows the `Alt-Svc` upgrade	HTTP/2 first, HTTP/3 from the second request
www.cloudflare.com	Cloudflare production	HTTP/2 first, HTTP/3 from the second request
www.litespeedtech.com	LiteSpeed's `lsquic`	HTTP/2 only; the QUIC handshake failed

Other major sites (`www.google.com`, `facebook.com`) serve HTTP/3 too, but in the UK and EU they may redirect

to a cookie-consent host first — which the client refuses unless you allow that host.

The "HTTP/2 first" pattern is normal: the JDK learns from the server's `Alt-Svc` header that HTTP/3 is available and uses it from the next request. Chapter 28 explains the discovery modes.

1.3. Requirements

Java	25 or later; HTTP/3 needs JDK 26 or later
Groovy	6.0 or later
Logging	Any SLF4J 2.x binding (the library depends on `slf4j-api` only)
Optional	`jackson-databind` 2.x, for Jackson-backed typed results

Chapter 2. Core Features and Capabilities

What the library covers, in one table, with where to read more. You need only the first few rows to be productive; the rest are there when your application needs them.

Capability	What you get	Read
Essentials		
Request DSL	`client.getSync('/orders') { query(…); header …; json(…) }` — every verb, sync and async	Chapter 9
Reading responses	Status, headers, text, bytes, JSON, XML, CSV; typed results such as `getSync(Order, '/orders/42')`	Chapter 13
Errors	`HttpResponseException` with status, headers and body as fields; RFC 9457 problem details; log-safe messages	Chapter 22
REST patterns	Pagination (page, offset, cursor, `Link` header), filtering and sorting, versioned APIs	Chapter 10
Sync, async, streaming	`getSync` waits, `get` returns a `CompletableFuture`; downloads and event streams never load the whole body	Chapter 14
Secure defaults	Certificate and host-name verification, host allow-listing on every redirect, bounded response sizes	Chapter 15
Content		
Codecs	JSON, XML, CSV, form and text built in; typed mapping onto your classes; your own encoders and parsers; optional Jackson	[codecs]
Response handlers	`response { success {}; when(404) {}; failure {}; exception {} }` turns outcomes into values	Section 20.1
Multipart	File uploads and form parts, from a DSL or a list	Section 11.6
Compression	Opt-in gzip and deflate; the size limit counts decompressed bytes	Chapter 26
Streaming	Downloads to files; uploads from files, streams and publishers; lines, Server-Sent Events, NDJSON	Chapter 25

Capability	What you get	Read
Connecting		
Protocols	HTTP/1.1, HTTP/2, and HTTP/3 on JDK 26+ with fallback	Chapter 28
TLS	Trust stores, client certificates (mutual TLS), protocol and cipher control	Section 17.4
Authentication	Basic, bearer, API key, OAuth2 client credentials	Chapter 23
Proxies	Explicit HTTP proxy with authentication and bypass rules	Chapter 24
Cookies	Automatic cookie handling with policies	Chapter 12
Operating		
Resilience	Retry policies with backoff and jitter; a circuit breaker you can inspect and listen to	Chapter 27
Metrics	Per-client counters: requests, outcomes, status classes, retries, timeouts, latency	Section 27.5
Interceptors	Middleware around every request: metrics, tracing, custom headers	Chapter 29
Logging	Credential-safe request lines through SLF4J	Chapter 31
Testing	A mock HTTP server, a test DSL and a Spock trait, published for your own tests	Chapter 7
URL builder	Build URLs with encoded query parameters	Chapter 30

2.1. What it deliberately does not do

- **SOCKS proxies.** The JDK `HttpClient` ignores them and connects directly; the client refuses them rather than bypass your proxy silently (Chapter 24).
- **Digest authentication.** Not supported by the JDK client and rare in modern APIs.
- **Server-Sent Event reconnection.** Events carry their `id`, so you can resume, but the client does not reconnect on its own.
- **Response caching.** Cache in your own code, or use an interceptor.

Chapter 3. History

1.x (2025). The first releases wrapped the JDK `HttpClient` in a Groovy DSL with retries, a circuit breaker, multipart uploads, cookie handling and a testing toolkit, running every request on a virtual thread.

2.0 (October 2026). A security review of 1.0.2 led to a release that changed several defaults: TLS certificates are verified by the plain constructor, redirects are checked against the allowed hosts on every hop, response sizes are enforced while the body arrives, and the synchronous methods throw the real exception rather than `ExecutionException`. It also moved to Groovy 6 and Java 25 and added HTTP/3. The request DSL was tightened: per-request settings such as `timeout(…)` take effect on every attempt, and an unknown call in a request closure fails instead of being ignored.

2.1 (this release). Content and connectivity: codecs and typed results, response handlers, error detail and problem details, streaming, compression, authentication helpers, explicit proxies, and the testing toolkit published as test fixtures. The DSL now resolves your own constants and variables correctly inside request closures, and statically compiled callers are supported. Nothing breaks for 2.0 callers who do not parse exception messages; see Chapter 34.

Chapter 4. Quick Start

This chapter takes you from nothing to a working, tested client in about fifteen minutes. Follow the steps in order; each one builds on the last and tells you what you should see.

4.1. Step 1: Check your tools

```
java -version      # 25 or later (26+ if you want HTTP/3)
groovy -version    # 6.0 or later -- only needed for the script route in step 2
```

Any IDE with Groovy support works; IntelliJ IDEA gives code completion inside the request closures (see Section 5.3).

4.2. Step 2: Get the library

Pick one route.

Route A — a Groovy script (fastest). Create `first.groovy`; the `@Grab` line downloads the library the first time you run it:

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
@Grab('org.slf4j:slf4j-simple:2.0.20')
import org.softwood.http.GroovyHttpClient
```

Route B — a Gradle project. In `build.gradle`:

```
plugins {
    id 'groovy'
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.apache.groovy:groovy:6.0.0'
    implementation 'io.github.woodmawa:groovy-httpclient:2.1.0'
    runtimeOnly 'ch.qos.logback:logback-classic:1.6.5'        // any SLF4J 2.x binding

    testImplementation testFixtures('io.github.woodmawa:groovy-httpclient:2.1.0')
    testImplementation 'org.spockframework:spock-core:2.5-groovy-6.0'
}

java {
    toolchain { languageVersion = JavaLanguageVersion.of(25) }
}
```

Route C — Maven.

```
<dependency>
    <groupId>io.github.woodmawa</groupId>
    <artifactId>groovy-httpclient</artifactId>
    <version>2.1.0</version>
</dependency>
```

The library declares only `slf4j-api`; without a binding you get a one-line SLF4J warning and no log output, which is harmless.

4.3. Step 3: Make your first request

Add to your script (or a `main` method):

```
def client = new GroovyHttpClient('https://httpbin.org')

def r = client.getSync('/get')
println r.statusCode          // 200
println r.json.url            // https://httpbin.org/get
println r.httpVersion         // HTTP_2
```

Run it (`groovy first.groovy`). You should see `200`, the URL and `HTTP_2`.

What happened: the client was created once for a **base URL**; `getSync` sent `GET /get`, waited for the answer and returned an `HttpClientResponse`; `r.json` parsed the JSON body into maps and lists. Certificates were verified and the response size was bounded, without you configuring anything.

4.4. Step 4: Add query parameters and headers

```
def r = client.getSync('/get') {
    query(lang: 'groovy', page: 1)
    header 'Accept', 'application/json'
}
println r.json.args           // [lang:groovy, page:1]
```

The closure after the path is the **request DSL**: you describe the request, the client builds and sends it. Everything you can configure on a request goes here. Section 5.3 explains how it works.

4.5. Step 5: Read JSON into your own class

```
class Post { Integer userId; Integer id; String title; String body }

def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')
Post post = api.getSync(Post, '/posts/1')
println post.title
```

Passing a class as the first argument asks for a **typed result**.

4.6. Step 6: Handle an error

```
import org.softwood.http.GroovyHttpClient.HttpResponseException
```

```
try {
    client.getSync('/status/404')
} catch (HttpResponseException e) {
    println "${e.statusCode}: ${e.message}"
}
```

Any status of 400 or more raises `HttpResponseException`, carrying the status, headers and body.

4.7. Step 7: Send data

```
def created = client.postSync('/post', null) {
    json([name: 'Alice', role: 'admin'])
}
println created.json.json     // httpbin echoes what it received: [name:Alice, role:admin]
```

`post`, `put` and `patch` take an optional body string as their second argument; passing `null` and building the body in the closure is the usual style.

4.8. Step 8: Go asynchronous

Drop `Sync` from the method name and you get a `CompletableFuture` immediately:

```
def pending = (1..5).collect { client.get('/delay/1') }      // five requests in flight
println pending*.join()*.statusCode                          // after about one second, not five
```

4.9. Step 9: Configure the client for real use

The constructor's defaults are safe; for an application you will usually want explicit timeouts and retries. Use the builder:

```
import java.time.Duration

def client = GroovyHttpClient.builder('https://api.example.com')
        .connectTimeout(Duration.ofSeconds(5))
        .requestTimeout(Duration.ofSeconds(15))
        .retryPolicy(new GroovyHttpClient.RetryPolicy.Config().tap {
            maxAttempts 3
            backoffExponential(200, 2_000, true)
            onIoExceptions(true); onTimeouts(true); on5xx(true)
        }.build())
        .auth { bearer System.getenv('API_TOKEN') }
        .build()
```

Create it once (at application start, or as a singleton bean) and share it.

4.10. Step 10: Test it without the network

With the test fixtures on the test classpath (step 2, route B), a Spock specification runs your code against a real local server:

```
import org.softwood.http.HttpClientTestTrait
import spock.lang.Specification
```

```
class FirstSpec extends Specification implements HttpClientTestTrait {
    def "reads a user"() {
        given:
        testServer.when('/users/1').respondWithJson([id: 1, name: 'Alice'])

        expect:
        createTestClient().getSync('/users/1').json.name == 'Alice'
    }
}
```

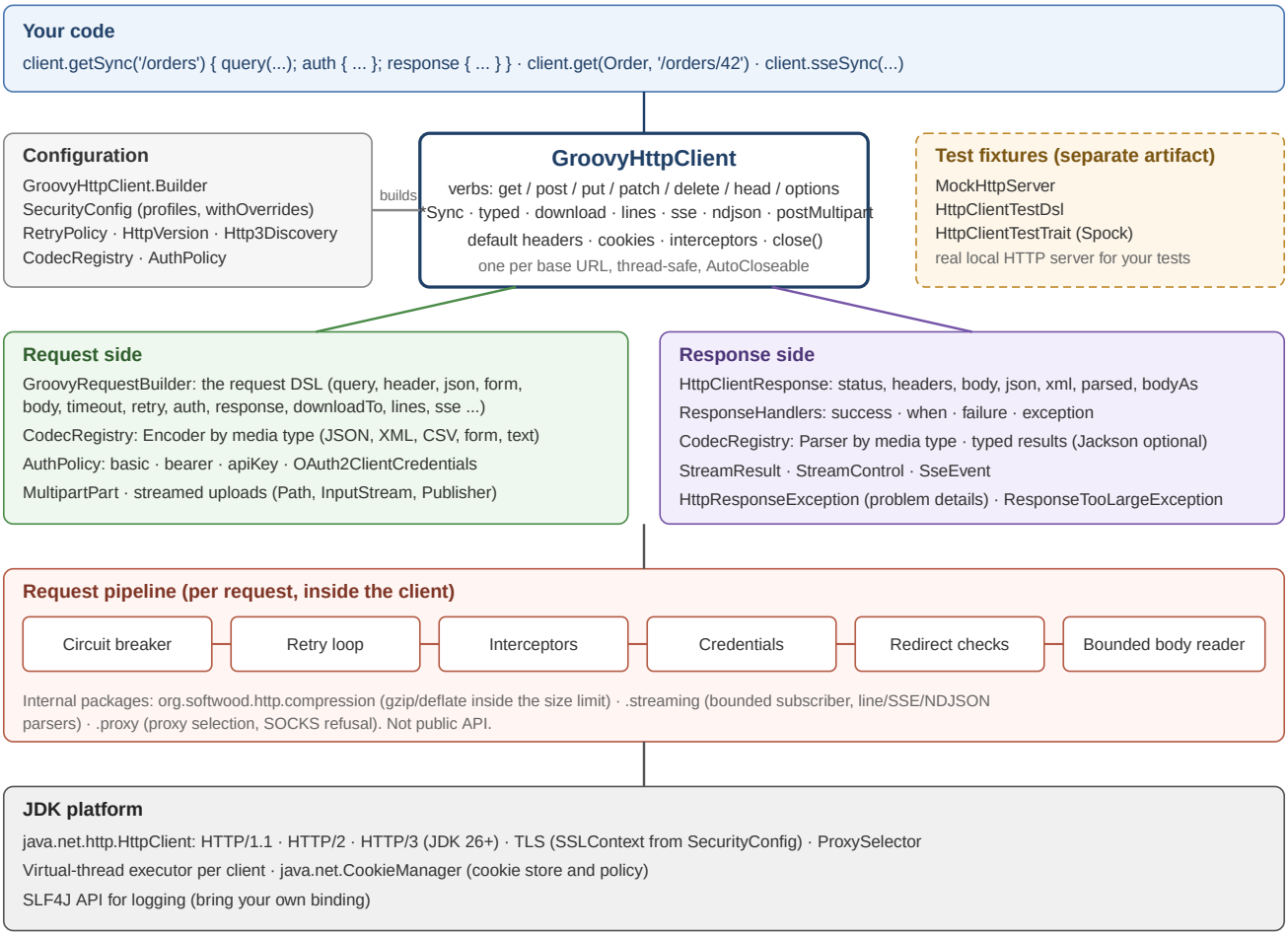
4.11. Before you go to production

- ☐ One client per base URL, created once and closed at shutdown.
- ☐ Timeouts chosen for the service you call (step 9).
- ☐ A retry policy if the service is occasionally flaky — and only for idempotent requests.
- ☐ Credentials from configuration or a secrets store, never in code.
- ☐ A trust store if the service uses a private CA (Chapter 15); never `trustAll()`.
- ☐ `maxResponseBytes` sized for your largest expected response, or streaming for big bodies.

Where next: Chapter 5 explains the architecture and the DSL; Part II walks through everyday use in more depth.

Chapter 5. How the Client Works

5.1. The core classes



You work with a handful of classes:

`GroovyHttpClient`	The client: one per base URL, holding the connection, TLS and protocol set-up, cookies, default headers and interceptors. Its verb methods (`get`, `postSync`, `download`…) send requests.
`GroovyHttpClient.Builder`, `SecurityConfig`	Two ways to configure a client: fluently, or as a settings object you can start from a profile and override.
`GroovyRequestBuilder`	The object behind the request closure — the request DSL. You rarely name it; you call its methods inside `{ }`.
`HttpClientResponse`	What a plain verb returns: status, headers, body, and parsed views of the body.
`CodecRegistry`	Converts request values to bytes and response bytes to values, by media type.
`ResponseHandlers`	The `response { }` DSL that turns outcomes into values.
`StreamResult`, `SseEvent`, `StreamControl`	The streaming results and controls.
`HttpResponseException` and friends	Failures, with all their detail as fields.

Underneath is the JDK's own `java.net.http.HttpClient`, so HTTP/2, HTTP/3, TLS and proxies are the platform's implementations; the library adds the DSL, the security policy, content handling, resilience and streaming around it.

5.2. The request pipeline

Each request passes through the same stages. Knowing the order explains most of the behaviour described later.

```
your request closure      headers, query, body, auth, response { }, per-request settings
circuit breaker           refuses at once while open
retry loop                attempts with backoff (never for one-shot uploads or delivered streams)
your interceptors         first added = outermost
credentials               added per attempt, to the base URL's origin only
redirect follower         every hop checked against allowed hosts; credentials removed across origins
JDK HttpClient            HTTP/1.1, HTTP/2 or HTTP/3; TLS; proxy
bounded body reader       decompress -> count against the size limit -> memory, file or your consumer
status check              400 and above become HttpResponseException
response handlers         success / when / failure / exception -> the value you get back
```

So retries see real 4xx/5xx responses, handlers run once on the final outcome, the size limit holds for compressed and streamed bodies alike, and credentials never follow a redirect to another site.

5.3. The DSL

The request DSL is the heart of the library. Compare the same request written against the JDK client directly:

```
// JDK HttpClient
def body = JsonOutput.toJson([sku: 'A1', qty: 2])
def query = 'tenant=' + URLEncoder.encode(tenant, UTF_8) + '&dryRun=true'
def request = HttpRequest.newBuilder(URI.create("https://api.example.com/orders?${query}"))
        .header('Content-Type', 'application/json')
        .header('Authorization', 'Bearer ' + token)
        .timeout(Duration.ofSeconds(10))
        .POST(HttpRequest.BodyPublishers.ofString(body))
        .build()
def response = jdkClient.send(request, HttpResponse.BodyHandlers.ofString())
if (response.statusCode() >= 400) throw new RuntimeException("HTTP ${response.statusCode()}")
def order = new JsonSlurper().parseText(response.body())
```

```
// Groovy HTTP Client
def order = client.postSync('/orders', null) {
    query(tenant: tenant, dryRun: true)
    json([sku: 'A1', qty: 2])
    timeout Duration.ofSeconds(10)
}.json
```

The second version says **what** the request is; encoding, headers, credentials, status checking and parsing follow from it.

How it works

A verb's last argument is a Groovy closure. The client creates a `GroovyRequestBuilder`, makes it the closure's **delegate** with the `DELEGATE_FIRST` strategy, and runs the closure. Inside the closure:

- calls such as `query(…)`, `header …` and `json(…)` go to the request builder, which records them;
- every other name — your local variables, fields, constants and methods — resolves to your own code

as usual;

- a call the builder does not recognise fails with `MissingMethodException`, so a typo is caught rather than silently ignored.

When the closure returns, the client turns what was recorded into an `HttpRequest` and sends it through the pipeline above.

```
static final String TENANT = 'acme'

List<String> openOrders(String status) {
    client.getSync('/orders') {
        query(tenant: TENANT, status: status)      // TENANT and status are yours
        header 'Accept', 'application/json'
    }.json*.id
}
```

The same pattern is used throughout, so once you know one block you know them all:

`client.getSync(path) { … }`	the request
`auth { … }`	credentials (client, builder or request)
`response { … }`	response handlers
`retry { … }`	a retry policy for one request
`codecs { … }`	content encoders and parsers, on the builder
`postMultipart(path) { b → b.part { … } }`	multipart parts

What it gives you

- **Readable requests.** A request reads as a description of the request; there is no builder chain to finish and no `send` to forget.
- **Defaults with local overrides.** Configure the client once; each request states only how it differs (a longer timeout, other credentials, a different retry policy). See Chapter 16.
- **IDE completion and type checking.** The verbs declare their closures with `@DelegatesTo(GroovyRequestBuilder)`, so IntelliJ IDEA completes request methods inside `{ }`, and `@TypeChecked` or `@CompileStatic` code checks them at compile time.
- **Fail-fast mistakes.** Unknown calls in a request closure, unknown settings in a `SecurityConfig` and invalid values all fail immediately, with a message naming the problem.
- **Plain Groovy inside.** A request closure is ordinary code: use `if`, loops and your own methods to build it.

  ```
  client.getSync('/search') {
      query(q: term)
      if (page) query(page: page)
      filters.each { k, v -> query((k): v) }
  }
  ```

- **Reuse without inheritance.** Put shared request settings in a method that takes the builder, and call

it with `delegate`:

```
static void tracing(GroovyHttpClient.GroovyRequestBuilder b) {
    b.header 'X-Request-Id', UUID.randomUUID().toString()
    b.header 'X-Client', 'orders-service'
}

client.getSync('/orders') { tracing(delegate); query(status: 'open') }
```

Settings that apply to every request belong on the client instead: default headers, `auth { }`, default response handlers and interceptors.

From Java

Java code uses the same client. The verbs without a closure, the builder and `SecurityConfig` are plain Java APIs; a request closure can be written as a Groovy `Closure` subclass, but if you configure many requests from Java, put the request-building in a small Groovy helper class.

```
GroovyHttpClient client = GroovyHttpClient.builder("https://httpbin.org").build();
GroovyHttpClient.HttpClientResponse r = client.getSync("/get");
System.out.println(r.getStatusCode());
```

5.4. Threads

Each client runs its requests on its own virtual-thread executor. The `*Sync` methods are the asynchronous ones followed by a wait, so it costs nothing to block. Response handlers and stream consumers run on the thread that completes the request; blocking inside them is safe.

A client is thread-safe and meant to be shared. Create one per base URL and close it (it is `AutoCloseable`) when the application stops.

Common Usage

Chapter 6. Public Test Sites

The examples in this part run against public services, so you can paste them into a Groovy script or console and see real responses.

Site	Good for	Try
httpbin.org	Echoes requests back as JSON. Status codes, redirects, cookies, basic and bearer auth, gzip and deflate, delays, streams.	`/get`, `/post`, `/status/404`, `/redirect/2`, `/cookies/set?flavour=oat`, `/basic-auth/user/pass`, `/gzip`, `/stream/5`, `/delay/2`
postman-echo.com	An alternative echo service.	`/get`, `/post`, `/headers`
jsonplaceholder.typicode.com	A fake REST API with realistic resources: good for typed results.	`/posts/1`, `/users`, `/todos?userId=1`
api.restful-api.dev	A REST API whose objects you can create, update and delete.	`/objects`, `/objects/7`
stream.wikimedia.org	A public Server-Sent Events stream of live Wikipedia edits. Send a descriptive `User-Agent` header, or it answers 403.	`/v2/stream/recentchange`

For HTTP/3, see the endpoints listed in Chapter 1.



Public services are shared, rate-limited and occasionally down. They are for trying things out; your own tests should use the published mock server instead (Chapter 32), which is fast, deterministic and works offline.

The default client allows requests only to its base URL's host. To call several of these sites, create one client per site — the normal pattern — or allow absolute URLs (Chapter 8).

Chapter 7. The Test Platform

Public test sites are good for trying things out. For your own code you need something you control: a server that returns exactly what each test needs, runs offline, and lets you check what your code sent. The library's **test platform** is that server, published so your tests can use it.

Tool	What it is	Use it for
`MockHttpServer`	A real HTTP server on a free local port, answering each method and path with the status, headers and body you give it, and recording every request.	Any test framework (JUnit, Spock, a script).
`HttpClientTestDsl`	A fluent layer over a mock server: `when('/users/1').respondWithJson(…)`, then `verify('/users/1').wasCalled()`.	Readable tests with less code.

Tool	What it is	Use it for
`HttpClientTestTrait`	A Spock trait: a shared server per specification and a ready-made client.	Spock specifications.

Add the platform to your tests (Gradle):

```
testImplementation testFixtures('io.github.woodmawa:groovy-httpclient:2.1.0')
```

Chapter 32 has the Maven coordinates and the full API.

7.1. Example 1: stub a service and check what your code sent

Suppose your code under test is a small service class:

```
class OrderService {
    final GroovyHttpClient client
    OrderService(GroovyHttpClient client) { this.client = client }

    Map place(String sku, int qty) {
        client.postSync('/orders', null) { json([sku: sku, qty: qty]) }.json
    }
}
```

A Spock test against `MockHttpServer`:

```
import org.softwood.test.MockHttpServer

class OrderServiceSpec extends Specification {
    MockHttpServer server = new MockHttpServer()
    def setup()   { server.init() }
    def cleanup() { server.shutdown() }

    def "places an order"() {
        given:
        def orders = server.addRequestCheck('POST', '/orders', 201)
                .withResponseBody('{"id":42,"status":"placed"}')
                .withResponseHeaders(['Content-Type': 'application/json'])
        def service = new OrderService(new GroovyHttpClient(SecurityConfig.testing
("http://localhost:${server.port}")))

        when:
        def result = service.place('A1', 2)

        then:
        result.id == 42
        orders.lastRequest.bodyText == '{"sku":"A1","qty":2}'
        orders.lastRequest.header('Content-Type').startsWith('application/json')
    }
}
```

The client is built with `SecurityConfig.testing(…)`: it allows the `http://localhost` URL, turns on logging, and puts the start of error bodies into exception messages so failures are easy to read.

7.2. Example 2: the same with the test DSL

```
import org.softwood.http.HttpClientTestDsl

def server = HttpClientTestDsl.create().start()
server.when('/orders').respondWithJson([id: 42, status: 'placed'], 201)

def service = new OrderService(server.createClient())
assert service.place('A1', 2).id == 42

server.verify('/orders')
      .wasCalled(1)
      .withMethod('POST')
      .withJsonBody { json -> json.sku == 'A1' && json.qty == 2 }
server.close()
```

7.3. Example 3: a Spock specification with the trait

```
import org.softwood.http.HttpClientTestTrait

class OrderServiceTraitSpec extends Specification implements HttpClientTestTrait {

    def setup() { resetTestServer() }          // each feature starts clean

    def "places an order"() {
        given:
        mockEndpoint('/orders') {
            status 201
            json([id: 42, status: 'placed'])
        }

        expect:
        new OrderService(createTestClient()).place('A1', 2).status == 'placed'
        verifyRequest('/orders').wasCalled(1).withMethod('POST')
    }
}
```

7.4. Example 4: test the unhappy paths

The cases that matter most are the ones a public site will not give you on demand:

```
def "a 503 is retried, then reported"() {
    given:
    def check = server.addRequestCheck('GET', '/orders/42', 503)
    def client = GroovyHttpClient.builder("http://localhost:${server.port}")
            .retryPolicy(new GroovyHttpClient.RetryPolicy.Config().tap {
                maxAttempts 3; backoffExponential(10, 20, false); on5xx(true)
            }.build())
            .build()

    when:
    client.getSync('/orders/42')

    then:
    def e = thrown(GroovyHttpClient.HttpResponseException)
    e.statusCode == 503
    check.received.size() == 3                 // the first attempt and two retries
}
```

```
def "a slow service hits the timeout"() {
    given:
    server.addRequestCheck('GET', '/slow', 200).withDelay(2_000)
    def client = new GroovyHttpClient(SecurityConfig.testing("http://localhost:${server.port}"))

    when:
    client.getSync('/slow') { timeout Duration.ofMillis(200) }

    then:
    thrown(java.net.http.HttpTimeoutException)
}

def "the session cookie is sent back"() {
    given:
    server.addRequestCheck('POST', '/login', 200).withResponseCookies([SESSION: 's-1'])
    server.addRequestCheck('GET', '/me', 200).withExpectedCookies([SESSION: 's-1']).withResponseBody
('{"user":"alice"}')
    def client = new GroovyHttpClient(SecurityConfig.testing("http://localhost:${server.port}"))

    when:
    client.postSync('/login', null) { form(user: 'alice') }

    then:
    client.getSync('/me').json.user == 'alice'   // 400 if the cookie were missing
}
```

7.5. Example 5: test asynchronous code

Asynchronous calls finish after the line that starts them, so a test must wait — but never with `Thread.sleep`, which is either too short (a flaky test) or too long (a slow suite). Use one of two tools.

When you hold the future: Groovy 6's `await`. Groovy 6 has `await` built in. It waits for a `CompletableFuture` and returns its value, and if the future failed it throws the **original** exception — not a `CompletionException` — so Spock's `thrown(…)` works directly:

```
def "fetches two orders concurrently"() {
    given:
    server.addRequestCheck('GET', '/orders/1', 200).withResponseBody('{"id":1}')
            .withResponseHeaders(['Content-Type': 'application/json'])
    server.addRequestCheck('GET', '/orders/2', 200).withResponseBody('{"id":2}')
            .withResponseHeaders(['Content-Type': 'application/json'])

    when:
    def (a, b) = await(client.get('/orders/1'), client.get('/orders/2'))    // both in flight together

    then:
    a.json.id == 1
    b.json.id == 2
}

def "an async failure surfaces as the real exception"() {
    given:
    server.addRequestCheck('GET', '/orders/9', 404)

    when:
    await client.get('/orders/9')

    then:
    thrown(GroovyHttpClient.HttpResponseException)
}
```

To bound the wait, give the future a timeout first: `await client.get('/orders/1').orTimeout(2, TimeUnit.SECONDS)` throws `TimeoutException` if it takes longer.

When there is no future: Awaitility. Some code starts work and returns without a future — a fire-and-forget audit call, an event stream consumer, a circuit breaker listener. Wait for the **effect** instead, with Awaitility (`testImplementation 'org.awaitility:awaitility:4.3.0'`):

```
import org.awaitility.Awaitility
import java.util.concurrent.CopyOnWriteArrayList
import static java.util.concurrent.TimeUnit.SECONDS

class AuditService {
    final GroovyHttpClient client
    AuditService(GroovyHttpClient client) { this.client = client }
    void record(String event) { client.post('/audit', null) { json([event: event]) } }    // does not wait
}

def "audit events are sent in the background"() {
    given:
    def audit = server.addRequestCheck('POST', '/audit', 202)

    when:
    new AuditService(client).record('login')

    then:
    Awaitility.await().atMost(5, SECONDS).until { audit.received.size() == 1 }
    audit.lastRequest.bodyText == '{"event":"login"}'
}

def "the circuit opening is reported"() {
    given:
    server.addRequestCheck('GET', '/fail', 500)
    def c = new GroovyHttpClient(SecurityConfig.testing("http://localhost:${server.port}").withOverrides
(failureThreshold: 2))
    def states = new CopyOnWriteArrayList()
    c.onCircuitStateChange { from, to -> states << to }

    when:
    2.times { c.get('/fail') }                // asynchronous: nothing waits for these

    then:
    Awaitility.await().atMost(5, SECONDS).untilAsserted {
        assert states == [GroovyHttpClient.State.OPEN]
    }
}
```

`until { condition }` polls the closure until it is true; `untilAsserted { assert … }` polls until the assertions pass and, on timeout, fails with the last assertion's message. Collect results in thread-safe collections (`CopyOnWriteArrayList`, `ConcurrentHashMap`), because the client's callbacks run on its own threads.



In Groovy 6, `await` is a language feature, so a statically imported `Awaitility.await()` is shadowed by it: `import static org.awaitility.Awaitility.await` followed by `await().atMost(…)` does not call Awaitility. Write `Awaitility.await()` in full, as above.

You have a future (`client.get(…)`)	`await future` — or `future.join()` if you prefer plain Java
You are waiting for a side effect (a request reaching the server, a callback, a listener)	`Awaitility.await().atMost(…).until { … }`

You need to assert a sequence of events	`Awaitility.await()…untilAsserted { assert … }`

7.6. Choosing between public sites and the test platform

Public test sites	Test platform
Learning the API, trying an idea, demos	Your automated tests
Real network, real TLS, real HTTP/3	Local, fast, offline, deterministic
You get what the site returns	You decide every status, header, body and delay
Rate-limited, occasionally down	Always there

The full reference — every method of the three tools — is in Chapter 32.

Chapter 8. Creating a Client

A client represents **one remote service**: its base URL, how to connect to it securely, and how to behave when it misbehaves. Creating it well is most of the configuration work you will do; individual requests then only say what is different about them.

8.1. Choosing how to create it

Way	Use it when	Example
`new GroovyHttpClient(url)`	Scripts, prototypes, tools, and services where the safe defaults suit you.	`new GroovyHttpClient('https://httpbin.org')`
`GroovyHttpClient.builder(url)…build()`	Application code. You set what matters in code that reads like the settings it makes.	`builder(url).requestTimeout(…).auth { … }.build()`
`new GroovyHttpClient(SecurityConfig.production(url))`	Settings that differ by environment, or come from configuration files: pick a profile, override from config.	`SecurityConfig.production(url).withOverrides(cfg)`
`new GroovyHttpClient(config, codecRegistry)`	Several clients sharing one set of custom codecs.	see [codecs]

The builder and `SecurityConfig` accept the same settings; the builder is the more readable in code, `SecurityConfig` the more convenient when settings are data.

8.2. The constructor

```
def client = new GroovyHttpClient('https://httpbin.org')
```

This uses `SecurityConfig.defaults(url)`: certificates and host names verified, requests allowed only to the base URL's host, redirects followed (up to 5, each checked), a 10 s connect timeout, a 30 s request timeout, a 10 MB response limit, and no retries. That is a good choice whenever a failed call can simply be reported — a script, a command-line tool, a batch job that is re-run.

8.3. The builder

For services, configure the client explicitly. Every setting has a default, so set only what you have a reason to:

```
def orders = GroovyHttpClient.builder('https://orders.internal.example.com')
        .connectTimeout(Duration.ofSeconds(3))        // fail fast if the service is down
        .requestTimeout(Duration.ofSeconds(10))       // its slowest normal response, plus margin
        .retryPolicy(standardRetries)                 // transient failures happen
        .failureThreshold(5).resetTimeoutMs(30_000)   // stop calling a service that is down
        .maxResponseBytes(5_000_000L)                 // its largest normal response, plus margin
        .compression(true)                            // it serves gzip
        .trustStore(internalCa)                       // its certificate comes from a private CA
        .auth { oauth2 { tokenUrl tokenEndpoint; clientId 'orders-ui'; clientSecret secret } }
        .build()
```

Ask these questions for each service you call, and set the matching builder method:

Question	Setting	Chapter
How quickly should I give up if it does not answer?	`connectTimeout`, `requestTimeout`	Section 17.2
Can it fail briefly, and is it safe to repeat a call?	`retryPolicy`	Chapter 27
What should happen when it is down for minutes?	`failureThreshold`, `resetTimeoutMs`	Section 27.2
How large can its responses be?	`maxResponseBytes`, or stream	Section 17.12
How does it authenticate me?	`auth { }`	Chapter 23
Does it use a private CA, or need my certificate?	`trustStore`, `keyManagers`	Section 17.4
Is there a proxy between us?	`proxy`, `noProxy`	Chapter 24
Does it serve compressed or HTTP/3 responses?	`compression`, `http3`	Chapter 26, Chapter 28
Which outcomes should never be exceptions?	`response { }`	Section 20.1

Chapter 17 lists every setting.

8.4. Profiles and externalised settings

`SecurityConfig` has named starting points:

`defaults(url)`	What the plain constructor uses (above).
`production(url)`	TLS 1.3 only, 5 s connect and 20 s request timeouts, 3 attempts with exponential backoff for I/O errors, timeouts and 5xx, metrics on.
`staging(url)`	`production` with request logging on and a 30 s request timeout.
`testing(url)`	For tests against local servers: absolute URLs allowed, all cookies accepted, logging on, no retries, metrics off, 120 characters of error body in exception messages. Certificates are still verified.
`trustAll(url)`	Accepts **any** certificate. Throwaway local debugging only; it logs a warning.

Start from the closest profile and override what differs. Because `withOverrides` takes a map, settings can come straight from your application's configuration:

```
// application.yml
// orders:
//   baseUrl: https://orders.internal.example.com
//   requestTimeout: PT10S
//   maxResponseBytes: 5000000

def cfg = appConfig.orders
def client = new GroovyHttpClient(SecurityConfig.production(cfg.baseUrl).withOverrides(
        requestTimeout  : Duration.parse(cfg.requestTimeout),
        maxResponseBytes: cfg.maxResponseBytes as long))
```

An unknown key is refused with the list of valid keys, so a typo in a configuration file fails at start-up rather than silently leaving a default in place.

8.5. Calling more than one host

By default a client refuses absolute URLs and any host other than its base URL's. Usually the right answer is one client per service. When one client genuinely needs several hosts — a CDN alongside an API, say — allow both explicitly:

```
def client = GroovyHttpClient.builder('https://api.example.com')
        .allowAbsoluteUrls(true)
        .allowedHosts(['api.example.com', 'files.example.com'] as Set)
        .build()

client.getSync('https://files.example.com/report.pdf')
```

This is the protection against server-side request forgery: a URL that came from user input cannot make your service call an arbitrary host.

8.6. Default headers

Headers that every request to this service needs:

```
client.withHeader('X-Client', 'orders-service')      // add a value
client.setHeader('Accept', 'application/json')       // replace all values
client.withHeaders(['X-Team': 'payments'])
client.clearHeaders()
client.defaultHeaders                                 // what is set now
```

A request can add to them with `header(…)` or replace one with `setHeader(…)` in its closure. Use default headers for things that never change per request (client identification, `Accept`); use `auth { }` for credentials, never a default `Authorization` header, so credentials get the origin and redirect protection described in Chapter 23.

8.7. Lifecycle

A client owns a connection pool and an executor. Create it once, share it, and close it when the application stops:

- **Scripts:** `withCloseable`.

```
new GroovyHttpClient('https://httpbin.org').withCloseable { client ->
    println client.getSync('/get').statusCode
}
```

- **Applications:** hold it in a field or a singleton and close it on shutdown.
- **Spring:** declare it as a bean; Spring calls `close()` for you.

```
@Bean
GroovyHttpClient ordersClient(OrdersProperties p) {
    GroovyHttpClient.builder(p.baseUrl).requestTimeout(p.timeout).auth { bearer { -> p.token } }.build()
}
```

Do not create a client per request: each one starts its own connection pool, so you lose connection reuse and HTTP/2 multiplexing.

Chapter 9. GET Requests

Most calls you make will be GETs: fetch a resource, search a collection, check something exists. This chapter shows how to build them and, as importantly, how to read what comes back.

9.1. Paths

The path is the verb's first argument and is appended to the base URL, including any path the base URL already has:

```
def api = GroovyHttpClient.builder('https://api.example.com/v2').build()
api.getSync('/orders/42')            // GET https://api.example.com/v2/orders/42
api.getSync("/customers/${id}/orders")
```

The path is used as you give it. If part of it comes from a user or from data, encode that segment first, so a value such as `a/b` or `x y` cannot change the path:

```
import java.nio.charset.StandardCharsets

String seg(String s) { URLEncoder.encode(s, StandardCharsets.UTF_8).replace('+', '%20') }

api.getSync("/products/${seg(sku)}")
```

9.2. Query parameters

`query(Map)` adds parameters and encodes them for you; call it as often as you like:

```
def client = new GroovyHttpClient('https://httpbin.org')

def r = client.getSync('/get') {
    query(search: 'groovy http', page: 2)          // ?search=groovy+http&page=2
    query(tag: ['a', 'b'])                         // repeated: &tag=a&tag=b
    query(debug: null)                             // null values are skipped
}
assert r.json.args.search == 'groovy http'
```

A query string already in the path is kept, so `getSync('/get?debug=true') { query(page: 2) }` sends both. Prefer `query(…)` for anything that varies: values are encoded exactly once, so `&`, `=`, `/`, `?` and non-ASCII characters arrive intact.

9.3. Headers

```
client.getSync('/get') {
    header 'Accept', 'application/json'
    header 'X-Request-Id', UUID.randomUUID().toString()
}
```

`header` adds a value to whatever the client already sends; `setHeader` replaces it for this request. Headers the client sends on every request are set once on the client (Chapter 8).

9.4. Reading the response

A plain verb returns an `HttpClientResponse`:

```
r.statusCode                 // 200
r.success                    // true for 2xx; also clientError, serverError
r.getHeader('Content-Type')  // first value, case-insensitive
r.headers                    // Map<String, List<String>>
r.contentType                // MediaType, e.g. application/json; charset=utf-8
r.body                       // the body as text, decoded with the response charset
r.bytes                      // the raw bytes
r.json                       // parsed JSON: Map / List
r.xml                        // parsed XML: GPathResult
r.parsed                     // parsed according to Content-Type
r.bodyAs(Order)              // mapped onto your class
r.httpVersion                // HTTP_1_1, HTTP_2 or HTTP_3
```

Which accessor to use:

`json`	You know the service returns JSON and want to navigate it with GPath: `r.json.items*.id`.
`bodyAs(Type)` or a typed verb	You have a class for the resource and want compile-time names and types.
`parsed`	The content type varies (JSON from one endpoint, CSV from another) or you are writing generic code.
`xml`	XML services; navigate with GPath: `r.xml.order.@id`.
`body` / `bytes`	You need the exact text or bytes: signatures, hashing, passing through unchanged.

`json` and `xml` throw `ContentParseException` if the body is not of that type; `jsonOrNull` and `xmlOrNull` return null instead. The body is held in memory with the response, so you can use several accessors on the same response.

9.5. Typed results

Map the body onto your own class:

```
class Post {
    Integer userId
    Integer id
    String title
    String body
}

def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

Post p = api.getSync(Post, '/posts/1')                // a typed verb
List<Post> all = api.getSync('/posts').json.collect { it as Post }
```

Nested objects and arrays of classes are mapped too. A JSON key your class has no property for fails with `ContentParseException` naming the class and the key, so a renamed field is caught rather than silently dropped. [codecs] explains the mapping rules and how Jackson takes over when it is on the classpath.

9.6. Conditional GET

Servers that send an `ETag` or `Last-Modified` header let you re-fetch only when something changed. A `304 Not Modified` is not an error, so it comes back as a response:

```
String etag = cache.etag
def r = client.getSync('/catalogue') { if (etag) header 'If-None-Match', etag }

if (r.statusCode == 304) {
    return cache.value                          // unchanged
}
cache.update(r.getHeader('ETag'), r.json)
```

9.7. HEAD and OPTIONS

```
def h = client.headSync('/get')             // headers only: size, type, last modified
println h.getHeader('Content-Length')

def o = client.optionsSync('/get')          // what the resource supports
println o.getHeader('Allow')
```

Collections, pagination, filters and versioned APIs are covered in Chapter 10.

Chapter 10. REST API Patterns

Three things almost every real API makes you deal with: collections that come back a page at a time, filters and sorting expressed in the URL, and versions. None needs special support in the client — each is a small pattern built from `query`, `header` and the response accessors — but it is worth getting them right once.

10.1. Pagination

APIs page collections in one of four ways. Find out which yours uses, then use the matching loop. Each example collects every item; in practice, stop as soon as you have what you need.

Page number and size (`?page=2&size=50`):

```
List<Map> all = []
int page = 1
while (true) {
    def items = client.getSync('/orders') { query(page: page, size: 50) }.json.items
    all.addAll(items)
    if (items.size() < 50) break              // a short page is the last page
    page++
}
```

Offset and limit (`?offset=100&limit=50`), often with a total:

```
int offset = 0
def first = client.getSync('/orders') { query(offset: 0, limit: 50) }.json
List<Map> all = new ArrayList<>(first.items)
int total = first.total
for (offset = 50; offset < total; offset += 50) {
    all.addAll(client.getSync('/orders') { query(offset: offset, limit: 50) }.json.items)
}
```

Cursor or continuation token (the response says where to continue):

```
List<Map> all = []
String cursor = null
do {
    def page = client.getSync('/orders') {
        query(limit: 100)
        if (cursor) query(cursor: cursor)
    }.json
    all.addAll(page.items)
    cursor = page.nextCursor
} while (cursor)
```

`Link` **header** (RFC 8288, as used by GitHub and many others): the next page's URL is in a response header. That URL is absolute, so the client must allow absolute URLs to its own host:

```
def gh = GroovyHttpClient.builder('https://api.github.com')
        .allowAbsoluteUrls(true)                          // allowedHosts stays [api.github.com]
        .build()

String nextLink(String header) {
    header?.split(',')?.find { it.contains('rel="next"') }?.with { it.substring(it.indexOf('<') + 1, it.indexOf
('>')) }
}

List<Map> repos = []
String url = '/orgs/apache/repos?per_page=100'
while (url) {
    def r = gh.getSync(url)
    repos.addAll(r.json)
    url = nextLink(r.getHeader('Link'))
}
```



To process items as they arrive rather than collect them all, wrap the loop in a method that takes a closure, or use the asynchronous verbs to fetch the next page while you process the current one. For very large exports, prefer an endpoint that streams NDJSON (Section 25.2) over thousands of pages.

10.2. Filtering and sorting

Filters and sort orders are query parameters; the only care needed is encoding, which `query(…)` does for you:

```
def r = client.getSync('/orders') {
    query(status: ['open', 'held'])                  // ?status=open&status=held
    query(customer: 'Smith & Sons')                  // & encoded, not a separator
    query(createdAfter: '2026-10-01T00:00:00Z')      // ':' encoded
    query(sort: '-createdAt,total')                  // the API's own syntax, passed as a value
    query(fields: 'id,status,total')                 // sparse field sets
}
```

Some APIs take a filter expression (`?filter=status==open;total>100` in RSQL, or `?$filter=` in OData). Build the expression as a string and pass it as one value; `query` encodes the operators so they survive the trip:

```
String filter = "status==open;total=gt=${minTotal}"
client.getSync('/orders') { query(filter: filter) }
```

When filters are optional, build the query with ordinary Groovy (see Section 5.3):

```
client.getSync('/orders') {
    criteria.findAll { k, v -> v != null }.each { k, v -> query((k): v) }
}
```

10.3. API versions

APIs carry their version in one of four places. Each maps onto a client feature:

Where	Example	With this client
Path	`/v2/orders`	Put the version in the base URL: `builder('https://api.example.com/v2')`. Every path is appended to it.
Header	`API-Version: 2026-10-01`	A default header: `client.setHeader('API-Version', '2026-10-01')`.
Media type	`Accept: application/vnd.acme.v2+json`	A default `Accept` header. The `+json` suffix means the built-in JSON codec parses it with no extra set-up.
Query	`?api-version=2026-10-01`	`query('api-version': …)` per request, or an interceptor if every request needs it.

```
// Path versioning: one client per major version you use
def v1 = GroovyHttpClient.builder('https://api.example.com/v1').build()
def v2 = GroovyHttpClient.builder('https://api.example.com/v2').build()

// Media-type versioning
def acme = GroovyHttpClient.builder('https://api.acme.example').build()
acme.setHeader('Accept', 'application/vnd.acme.v2+json')
assert acme.getSync('/orders/42').json.id == 42          // parsed as JSON via the +json suffix
```

Migrating between versions: create a client per version and move calls across one at a time. Because the

version lives in the client, the request code itself does not change.

Chapter 11. Sending Data

11.1. The verbs and what they mean

Verb	Use it to	Retried by default?
`POST`	Create something, or run an action (`/orders`, `/orders/42/cancel`). Not idempotent: sending it twice may create two orders.	No
`PUT`	Replace a resource with the representation you send. Idempotent: sending it twice leaves the same result.	Yes, if a retry policy is set
`PATCH`	Change part of a resource. Usually not idempotent.	No
`DELETE`	Remove a resource. Idempotent.	Yes, if a retry policy is set

`post`, `put` and `patch` take a path, an optional body string, and the request closure. Usually you pass `null` and build the body in the closure:

```
def client = new GroovyHttpClient('https://httpbin.org')

client.postSync('/post', null) { json([name: 'Alice', roles: ['admin']]) }
client.putSync('/put', null)   { json([id: 7, name: 'Alice']) }
client.patchSync('/patch', null) {
    body([name: 'Alicia'], 'application/merge-patch+json')     // RFC 7396 merge patch
}
client.deleteSync('/delete')

client.postSync('/post', 'plain text body')                  // a string body, sent as is
```

11.2. Choosing how to send the body

Method	Sends	Use it for
`json(value)`	`application/json`	Most APIs. Maps, lists and your objects are serialised; a `String` is sent unchanged (pre-built JSON).
`form(Map)`	`application/x-www-form-urlencoded`	HTML-style forms, OAuth token endpoints, older APIs. A list value repeats the field.
`body(value, type)`	the type you name	Any other format with a codec: CSV, XML, your own media types ([codecs]).
`body(byte[], type)`	the bytes, unchanged	Binary payloads you already hold: images, PDFs, signed documents.

Method	Sends	Use it for
`body(Path)` / `body(File)`	the file, streamed	Uploads larger than you want in memory; replayable, so retries and 307/308 redirects work.
`body(InputStream, type)`	the stream, chunked	Data produced on the fly; sent once, never retried.
multipart	`multipart/form-data`	Uploads that combine files and fields, as browsers do.

11.3. JSON

```
class NewOrder { String sku; int qty }

client.postSync('/post', null) { json(new NewOrder(sku: 'A1', qty: 2)) }    // {"sku":"A1","qty":2}
client.postSync('/post', null) { json([items: [[sku: 'A1'], [sku: 'B7']]]) }
client.postSync('/post', null) { json('{"raw":"json"}') }                    // sent as given
```

With Jackson on the classpath, objects are serialised by Jackson, so your Jackson annotations apply.

11.4. Forms

```
def r = client.postSync('/post', null) {
    form(username: 'alice', remember: 'true', scope: ['read', 'write'])
}
assert r.json.form.username == 'alice'
```

11.5. Other content types

```
client.postSync('/post', null) { body([[sku: 'A1', qty: 2], [sku: 'B7', qty: 1]], 'text/csv') }   // header row + rows
client.postSync('/post', null) { body({ order(id: 7) { line(sku: 'A1') } }, 'application/xml') }  // markup builder
client.postSync('/post', null) { body(pdfBytes, 'application/pdf') }
```

11.6. Multipart documents

Multipart (`multipart/form-data`) is how browsers upload files: one request carrying several **parts**, each with a name, optionally a filename and its own content type. Use it when the server expects a form upload; for an API that accepts a raw file, `body(Path)` is simpler and streams.

Describe the parts in a DSL:

```
def r = client.postMultipartSync('/post') { b ->
    b.part {
        name 'description'
        content 'Quarterly report'
    }
    b.part {
        name 'file'
        filename 'report.csv'
```

```
        contentType 'text/csv'
        data new File('report.csv').bytes
    }
    b.header 'X-Upload-Id', 'u-42'
}
```

or build the parts yourself, which suits parts computed from data:

```
import org.softwood.http.MultipartPart

def parts = [
        MultipartPart.text('description', 'Quarterly report'),
        MultipartPart.json('metadata', '{"quarter":"Q3"}'),
        MultipartPart.bytes('file', 'report.csv', 'text/csv', csvBytes)
]
client.postMultipartSync('/post', parts) { header 'X-Upload-Id', 'u-42' }
```

With a list of parts, the closure is the ordinary request closure, so `auth { }`, `gzipBody()` and the other request settings work there. Part names, filenames and content types are checked for line breaks, so user-supplied file names cannot inject headers.

Fields and files. A part with a filename is a file upload; a part without one is a plain form field, which is how servers tell them apart (RFC 7578; for example Spring's `@RequestParam` versus `MultipartFile`):

Sent as a field (no filename)	Sent as a file
`MultipartPart.text(…)`, `MultipartPart.json(…)`	`MultipartPart.bytes(name, filename, …)`
DSL part with `content` and no `filename`	DSL part with a `filename`, or with `data` (named `<name>.bin` if you give no filename)

11.7. Redirects after a POST

Redirects are followed automatically (up to 5 hops, each checked against the allowed hosts). What happens to the method and body depends on the status:

`303 See Other`	The next request is a `GET` without a body: the usual "post, then redirect to the result" pattern.
`301`, `302`	A `POST` becomes a `GET` without a body, as browsers do.
`307`, `308`	Method and body are kept and re-sent to the new location. A one-shot body (an `InputStream` or publisher) cannot be re-sent, so the redirect is refused with `RedirectRejectedException`.

```
def r = client.postSync('/redirect-to?url=/get&status_code=303', null) { form(a: '1') }
assert r.json.url == 'https://httpbin.org/get'       // followed with a GET
```

To receive the 3xx response itself — to read a `Location` header after creating something, say — build the client with `allowRedirects(false)`.

11.8. Making POST safe to retry

POST and PATCH are never retried unless you say so, because a retry after a lost response could apply

the change twice. If the API supports idempotency keys, send one and opt in:

```
client.postSync('/payments', null) {
    header 'Idempotency-Key', UUID.randomUUID().toString()
    json(payment)
    retry { maxAttempts 3; onIoExceptions(true); onTimeouts(true); retryNonIdempotent(true) }
}
```

Chapter 12. Cookies

Most APIs authenticate with tokens and never set a cookie. Cookies matter when you talk to services built for browsers: session-based logins, load balancers that pin you to a server with a cookie, CSRF protection, and web pages you automate. This chapter covers how the client handles them and how to take control when you need to.

12.1. How the client handles cookies

Each client has its own cookie store (a `java.net.CookieManager`). It works like a browser's:

- A `Set-Cookie` header in a response stores the cookie, subject to the client's **cookie policy**.
- Later requests send back every stored cookie whose domain, path and `Secure` flag match the request, in a `Cookie` header.
- Cookies expire as their `Max-Age` or `Expires` says; session cookies (neither attribute) last as long as the client.
- The store is shared by all requests on that client and is thread-safe. It is not shared between clients, and it is not saved anywhere: a new client starts empty.

```
def client = new GroovyHttpClient('https://httpbin.org')

client.getSync('/cookies/set?flavour=oat')      // the server sets a cookie (and redirects)
def r = client.getSync('/cookies')              // the client sends it back
assert r.json.cookies.flavour == 'oat'
```

12.2. A session login

The typical reason to care: an application that logs you in with a form and then recognises you by a session cookie.

```
def app = GroovyHttpClient.builder('https://legacy.example.com').build()

// 1. log in: the response sets JSESSIONID (or similar); the client stores it
app.postSync('/login', null) { form(username: user, password: password) }

// 2. every later request carries the session cookie automatically
def orders = app.getSync('/api/orders').json

// 3. log out, or simply discard the client
app.postSync('/logout', null)
app.clearCookies()
```

Use one client per logged-in identity: two users sharing a client would share a session.

If the application uses a CSRF token in a cookie (often `XSRF-TOKEN`) and expects it echoed in a header, read it from the store:

```
String csrf = app.getCookie('XSRF-TOKEN')?.value
app.postSync('/api/orders', null) {
    header 'X-XSRF-TOKEN', csrf
    json(order)
}
```

12.3. Setting and reading cookies yourself

```
client.addCookie('session', 'abc123')            // stored, sent with later requests
client.addCookie('pref', 'dark', '/settings')    // only for paths under /settings
client.addCookie(new HttpCookie('lang', 'en'))   // full control: domain, max age, secure...
client.getCookie('session')?.value               // java.net.HttpCookie
client.cookies                                   // List<HttpCookie>
client.removeCookie('pref')
client.clearCookies()
```

For a cookie on one request only, set it in the request closure; it is not stored:

```
client.getSync('/cookies') {
    cookie 'trace', 't-1'
    cookies(ab: 'variant-b', region: 'eu')
}
```

Which to use:

The server sets it and expects it back	Nothing: automatic
You obtained a session value elsewhere (an SSO hand-off, a stored session)	`addCookie`
A value that varies per request (A/B variant, trace id)	`cookie(…)` in the request
Starting a fresh session	`clearCookies()`, or a new client

12.4. Cookie policy

The policy decides which `Set-Cookie` headers are stored:

`CookiePolicy.ACCEPT_ORIGINAL_SERVER`	The default. Accept a cookie only for the host that sent it. Right for nearly everything.
`CookiePolicy.ACCEPT_ALL`	Accept every cookie, including those for other domains. Used by the `testing()` profile, where local servers make domain checks awkward.
`CookiePolicy.ACCEPT_NONE`	Store nothing. For stateless API clients where a stray cookie (from a load balancer, say) must not influence later requests.

```
GroovyHttpClient.builder(url).cookiePolicy(CookiePolicy.ACCEPT_NONE).build()
client.cookiePolicy = CookiePolicy.ACCEPT_ALL          // or change it later
```

12.5. Cookies and security

- **Validation.** Names and values you add are checked against RFC 6265: a value containing `;`, `,`, whitespace, quotes, a backslash or control characters is refused, so data from a user cannot smuggle extra cookies. Encode such values (URL or Base64) first.
- **Redirects.** When a redirect leads to another origin, the `Cookie` header is removed along with credentials.
- **Secure cookies.** A cookie marked `Secure` is only sent over HTTPS.
- **Logging.** Cookie headers are never logged.

12.6. Testing code that uses cookies

`MockHttpServer` can set cookies and insist on receiving them:

```
server.addRequestCheck('POST', '/login', 200).withResponseCookies([JSESSIONID: 's-1'])
server.addRequestCheck('GET', '/api/orders', 200)
        .withExpectedCookies([JSESSIONID: 's-1'])          // answers 400 without it
        .withResponseBody('[]')
```

Chapter 13. Handling Results

Every call ends in one of three ways: a response below 400, a response of 400 or above, or no response at all (the connection failed, it timed out, the circuit breaker refused). The library gives you three styles for dealing with them. Pick per call, or mix them.

Style	Looks like	Best when
Exceptions (the default)	`try { client.getSync(…) } catch (HttpResponseException e)`	Failure is exceptional, and the caller further up should deal with it — most service-to-service calls.
Response handlers	`response { success {…}; when(404) {…}; exception {…} }`	Some non-2xx outcomes are normal answers (404 "not found", 409 "already exists"), or you want a fallback value.
Typed results	`Order o = client.getSync(Order, '/orders/42')`	You want your own type back. Combines with either style above.

13.1. Exceptions

A status of 400 or above throws `HttpResponseException` after any retries; network failures throw the underlying exception:

```
import org.softwood.http.GroovyHttpClient.HttpResponseException

def client = new GroovyHttpClient('https://httpbin.org')

try {
    client.getSync('/status/404')
} catch (HttpResponseException e) {
```

```
    println e.message           // HTTP 404 Not Found (GET https://httpbin.org/status/404, 0 bytes)
    println e.statusCode        // 404
    println e.responseBody      // the full body, if any
    println e.header('Date')    // response headers
} catch (java.net.ConnectException | java.net.http.HttpTimeoutException e) {
    println "service unreachable: ${e.class.simpleName}"
}
```

The message is short and safe to log: it never contains the response body or the query string, which often carry personal data or tokens. Everything else is a field on the exception (Chapter 22). The `*Sync` methods throw the real exception, never an `ExecutionException` wrapper.

A common pattern is translating HTTP failures into your own exceptions at the edge of your code:

```
Order fetchOrder(String id) {
    try {
        client.getSync(Order, "/orders/${id}")
    } catch (HttpResponseException e) {
        if (e.statusCode == 404) throw new OrderNotFound(id)
        throw new OrderServiceFailure("orders service: HTTP ${e.statusCode}", e)
    }
}
```

13.2. Response handlers

When some outcomes are normal, say so in the request and get a value back instead of an exception:

```
def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

Post post = api.getSync(Post, '/posts/9999') {
    response {
        success   { r -> r.bodyAs(Post) }
        when(404) { r -> null }                        // "not found" is an answer, not an error
        failure   { r -> throw r.toException() }       // any other status >= 400
        exception { Throwable t -> null }              // network failure, timeout...
    }
}
assert post == null
```

Use the typed verb (here `getSync(Post, …)`) to receive what a handler returns. Handlers can also be set once on the client for every request — for example to turn every 401 into your own `SessionExpired` exception. Section 20.1 covers status ranges, client-wide handlers and the precedence rules.

13.3. Typed results

Typed verbs combine with both styles: without handlers, a 2xx body is mapped onto your type and a failure throws as usual; with handlers, the handler's value is returned.

```
Order order = client.getSync(Order, '/orders/42')                    // throws on 404
Order maybe = client.getSync(Order, '/orders/42') { response { when(404) { null } } }
```

13.4. Problem details

Many APIs report errors as RFC 9457 problem details (`application/problem+json`). They are parsed for you,

in both styles:

```
catch (HttpResponseException e) {
    if (e.problem) {
        println "${e.problem.title}: ${e.problem.detail}"
        println e.problem.extensions.balance          // extra members
    }
}

response { when(422) { r -> throw new ValidationFailed(r.problem.detail) } }
```

13.5. Which to choose

- Calling a service your code depends on? Let exceptions propagate and translate them at one boundary.
- Looking something up that may not exist? `when(404) { null }`.
- Serving cached or default data while a dependency is down? `exception { … }` plus `when(500..599) { … }`.
- The same rule for every request to a service (401 means re-login)? A client-wide handler.

Chapter 14. Sync, Async and Streaming

Every request can be made three ways. They differ in **when you get control back** and **where the body goes**:

	You get back	The body
Sync (`getSync`)	The result, when it is complete	In memory
Async (`get`)	A `CompletableFuture`, immediately	In memory
Streaming (`download`, `lines`, `sse`, `ndjson`)	A `StreamResult` (or its future) when the body has been delivered	To a file, a stream or your callback, piece by piece

14.1. Synchronous: the default choice

```
def r = client.getSync('/orders/42')
```

Because every request runs on a virtual thread, blocking is cheap: a sync call in a web request handler, a batch job or a script ties up no platform thread. Use sync calls unless you have one of the reasons below to do otherwise; the code is simpler and stack traces are clearer.

`*Sync` methods wait at most 30 seconds by default; pass a `Duration` as the last argument to change that. The request's own timeout (Section 17.2) bounds the request separately.

```
def r = client.getSync('/reports/annual', null, Duration.ofMinutes(2))
```

14.2. Asynchronous: for concurrency and composition

The same verbs without `Sync` return a `CompletableFuture` at once. Use them when:

- **you call several services, or one service many times, and the calls are independent** — start them all, then wait once;
- **you compose calls** — the second depends on the first;
- **your framework is asynchronous** — return the future rather than blocking on it.

```
def client = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

// fan out: ten requests in flight together
def futures = (1..10).collect { id -> client.get("/posts/${id}") }
def titles = futures*.join().collect { it.json.title }

// compose: the second request needs the first's answer
client.get('/posts/1')
      .thenCompose { r -> client.get("/users/${r.json.userId}") }
      .thenAccept { r -> println "Written by ${r.json.name}" }
      .join()

// different services at once
def (user, orders) = [users.get('/users/7'), orders.get('/orders?user=7')]*.join()
```

A failed future completes exceptionally with the same exception the sync method would throw; `join()` wraps it in `CompletionException`, so prefer handling failures in the chain (`exceptionally`, `handle`) or with a `response { exception {…} }` handler.



Limit how many requests you start at once when fanning out over a large list — the server, not the client, is usually the bottleneck. A `java.util.concurrent.Semaphore` around each call is enough.

14.3. Streaming: for big or endless bodies

Streaming verbs never hold the whole body in memory. Use them when:

- **the body is large** — exports, backups, media: `download` writes it to a file;
- **the body never ends, or arrives over time** — event feeds, log tails, AI token streams: `sse`, `lines`, `ndjson` hand you each item as it arrives;
- **you want to start work before the last byte arrives.**

```
def bin = new GroovyHttpClient('https://httpbin.org')

// a download, written to a temporary file and moved into place when complete
bin.downloadSync('/image/png', Path.of('image.png'))

// newline-delimited JSON, one parsed value at a time
bin.ndjsonSync('/stream/5') { Map record -> println record.id }

// Server-Sent Events from a long-running stream: stop after ten
def wiki = new GroovyHttpClient('https://stream.wikimedia.org')
wiki.sseSync('/v2/stream/recentchange', { header 'User-Agent', 'my-app/1.0 (me@example.com)' }) { SseEvent e,
StreamControl c ->
    println e.data.take(80)
```

```
    if (c.items == 10) c.stop()
}
```

Streaming verbs come in sync and async forms too (`download` / `downloadSync`, `sse` / `sseSync`…). Uploads stream as well: `body(Path)` sends a file without loading it. Chapter 25 covers limits, timeouts, stopping and retry rules.

14.4. Decision guide

Situation	Use
One call, result needed now (web handler, job step, script)	`getSync`
Several independent calls	`get` … then `join`
A call that depends on another	`get(…).thenCompose { … }`
An asynchronous framework	`get`, returning the future
A response bigger than you want in memory	`download` / `downloadSync`
Events, logs or records arriving over time	`sse`, `lines`, `ndjson`
A large upload	`body(Path)` or `body(InputStream, type)` in any verb

Chapter 15. Security

An HTTP client sits on a trust boundary: it sends your credentials and data out, and lets other systems' data in. The library is built so that the safe behaviour is the default and the unsafe one needs a deliberate, visible decision.

15.1. What the client protects you from

Threat	What could happen	What the client does
Interception (man in the middle)	Someone between you and the server reads or alters traffic, including tokens.	Certificates and host names are verified on every TLS connection; TLS 1.2+ only (`production()`: 1.3). Turning verification off takes `trustAll()` and logs a warning.
Server-side request forgery (SSRF)	A URL from user input makes your service call an internal system (`http://169.254.169.254/...`).	Absolute URLs are refused by default; requests and every redirect hop may only reach allowed hosts.
Credential leakage	A redirect, a log line or an exception message carries a token somewhere it should not go.	Credentials go only to the base URL's origin and are removed on cross-origin redirects; HTTPS never redirects to HTTP; logs and messages never contain headers, bodies or query strings.

Threat	What could happen	What the client does
Resource exhaustion	A huge, endless or highly compressed ("zip bomb") response exhausts memory.	Bodies are limited (10 MB by default), counted as they arrive and after decompression; stream items are limited separately.
Injection	A value from a user adds headers or cookies (`"a\r\nX-Admin: true"`, `"x; admin=1"`).	Cookie values, multipart headers and other header-bound values are validated.
XML attacks (XXE, billion laughs)	A malicious XML response reads local files or explodes in memory.	The XML parser refuses DOCTYPE declarations and external entities.
Silent misconfiguration	A misspelt setting leaves an unsafe default in place.	Unknown settings are refused when the client is built.

You get all of this from `new GroovyHttpClient(url)`. What you configure is mostly **who to trust**: which certificates, which hosts.

15.2. Setting up security in development

The temptation in development is to switch verification off. Don't: it hides the configuration you will need in production, and `trustAll()` code has a way of reaching production. Instead, give your development set-up real certificates. The library ships scripts that make this a few commands.

The SSL tools

Six scripts — a `.sh` for Linux and Mac and a `.bat` for Windows of each — live in `src/main/resources/ssl-tools/` in the source repository and are packaged in the library jar under `ssl-tools/`. To get them from the jar:

```
jar xf ~/.m2/repository/io/github/woodmawa/groovy-httpclient/2.1.0/groovy-httpclient-2.1.0.jar ssl-tools/
chmod +x ssl-tools/*.sh
```

`create_keystore`	Generate a PKCS12 keystore with a key pair and a self-signed certificate — the server side of a local HTTPS set-up.
`import_certificate`	Import a certificate (a CA's, or a self-signed server's) into a PKCS12 **truststore** — what the client is told to trust.
`convert_pem_to_p12`	Turn a PEM certificate and private key (from OpenSSL, mkcert or your PKI team) into a PKCS12 keystore — for a client certificate (mutual TLS).

All three need `keytool` (part of the JDK); `convert_pem_to_p12` also needs `openssl`. Run any of them with `--help` for the full options.

Scenario 1: a local HTTPS server with a self-signed certificate

You run a service locally over HTTPS and want the client to trust it properly.

```
# 1. the server's keystore: key pair + self-signed certificate for localhost
./create_keystore.sh --type development --output dev-server.p12 --password changeit --hostname localhost
```

```
# 2. export the server certificate
keytool -exportcert -keystore dev-server.p12 -storepass changeit -alias dev-cert -rfc -file dev-server.crt

# 3. a truststore for the client containing that certificate
./import_certificate.sh --cert dev-server.crt --truststore dev-truststore.p12 --password changeit
```

Point your local server at `dev-server.p12` (in Spring Boot: `server.ssl.key-store`), and the client at the truststore:

```
import java.security.KeyStore

KeyStore loadPkcs12(String file, String password) {
    def ks = KeyStore.getInstance('PKCS12')
    new File(file).withInputStream { ks.load(it, password.toCharArray()) }
    ks
}

def client = new GroovyHttpClient(SecurityConfig.testing('https://localhost:8443')
        .withOverrides(trustStore: loadPkcs12('dev-truststore.p12', 'changeit')))
```

Certificates and host names are verified exactly as in production; only the trusted certificate differs.

Scenario 2: your organisation's private CA

Internal services use certificates from a company CA that the JDK does not know. Import the CA certificate once:

```
./import_certificate.sh --cert company-root-ca.crt --truststore company-truststore.p12 --password "$TRUST_PASS"
```

```
def client = GroovyHttpClient.builder('https://orders.internal.example.com')
        .trustStore(loadPkcs12('company-truststore.p12', System.getenv('TRUST_PASS')))
        .build()
```

Trusting the CA (rather than each server) means every internal service it issues for works without further steps. If you are behind an SSL-inspecting proxy, import the proxy's CA the same way (see the SSL certificate guide, `SSL_CERTIFICATE_GUIDE.md`).

Scenario 3: a client certificate (mutual TLS)

The service requires you to present a certificate. Your PKI team (or `mkcert`, or OpenSSL) gives you a PEM certificate and key:

```
./convert_pem_to_p12.sh --input client-cert.pem --key client-key.pem --output client.p12 --password "
$CLIENT_PASS"
```

```
import javax.net.ssl.KeyManagerFactory

def kmf = KeyManagerFactory.getInstance(KeyManagerFactory.defaultAlgorithm)
kmf.init(loadPkcs12('client.p12', System.getenv('CLIENT_PASS')), System.getenv('CLIENT_PASS').toCharArray())

def client = GroovyHttpClient.builder('https://partner.example.com')
        .keyManagers(kmf.keyManagers)
        .trustStore(loadPkcs12('partner-ca.p12', System.getenv('TRUST_PASS')))
```

```
        .build()
```

From development to production

Development	`SecurityConfig.testing(url)` plus a development truststore. Verification on, logging on, no retries.
CI	The same, with the truststore and passwords from CI secrets.
Staging	`SecurityConfig.staging(url)`; the organisation truststore; logging on.
Production	`SecurityConfig.production(url)`; TLS 1.3; truststore and passwords from your secrets store; no logging of requests unless you need it.

The code is the same in every environment; only the profile and the stores change — which is the point of not using `trustAll()` in development.

Good habits:

- Keep keystores and truststores out of version control (add `*.p12` to `.gitignore`), and passwords in environment variables or a secrets manager.
- Give development certificates short validity (`--validity 90`) and the right host name (`--hostname`).
- Use a separate truststore per environment rather than one that trusts everything.

15.3. Settings you will change

- **Who to trust:** `trustStore(…)` (above).
- **Bigger responses:** `maxResponseBytes(100_000_000L)` on the builder, `maxResponseBytes n` on one request — or download to a file instead.
- **More hosts:** `allowAbsoluteUrls(true)` and `allowedHosts([…] as Set)` (Chapter 8).
- **Stricter TLS:** `allowedTlsProtocols(['TLSv1.3'])`, `allowedCipherSuites([…])` (Section 17.4).



`SecurityConfig.trustAll(url)` accepts any certificate. Use it only for throwaway local debugging; it logs a warning every time a client is created with it.

Configuration

Chapter 16. The Configuration Model

Configuration lives in two places:

```
def client = GroovyHttpClient.builder('https://api.example.com')   // client configuration:
        .requestTimeout(Duration.ofSeconds(20))                    //   applies to every request
        .auth { bearer token }
        .build()

client.getSync('/reports/42') {                                    // request configuration:
    timeout Duration.ofSeconds(120)                                //   this request only,
    auth { bearer reportToken }                                    //   wins over the client's
}
```

- **Client configuration** is fixed when the client is built: connection, TLS, protocol, proxy, limits, defaults. It comes from the builder or from a `SecurityConfig` (a profile plus `withOverrides`); both accept the same settings. A few client-wide defaults can also be changed later on the client itself: default headers, cookies, `auth { }`, `defaultResponse { }` and interceptors.
- **Request configuration** is the closure passed to a verb. Where a request setting has a client counterpart (timeout, retry policy, auth, compression, size limit, response handlers), the request's value wins for that request. Response handlers merge key by key: a request's `when(404)` keeps the client's `when(401)`.

```
// The same settings, two ways
def a = GroovyHttpClient.builder(url).maxResponseBytes(5_000_000L).compression(true).build()
def b = new GroovyHttpClient(SecurityConfig.defaults(url).withOverrides(maxResponseBytes: 5_000_000L,
compression: true))
```

Settings are validated when the client is built. An unknown name in a `SecurityConfig` map is refused with the list of valid names, so a typo in a security setting cannot be silently ignored.

Chapter 17. Client Configuration

Client configuration decides how a client connects to its service and how it behaves when things go wrong. It is grouped into areas below. Each area says **what it controls**, **when to change it**, its settings — the builder method, the `SecurityConfig` key where it differs, and the default — and an example.

Area	Controls
Section 17.1	Which URLs the client may call
Section 17.2	How long it waits
Section 17.3	HTTP/1.1, HTTP/2 or HTTP/3
Section 17.4	Who it trusts and how it proves its own identity
Section 17.5	The route to the service

Area	Controls
Section 17.6	Whether and how far it follows redirects
Section 17.7	Which cookies it keeps
Section 17.8	How bodies are encoded, parsed and compressed
Section 17.9	The credentials it sends
Section 17.10	Headers and response handling for every request
Section 17.11	Retries and the circuit breaker
Section 17.12	Size limits
Section 17.13	Logging and metrics
Section 17.14	Ready-made combinations

17.1. Base URL and allowed hosts

Controls: where requests can go — the base every path is appended to, and the hosts that requests and redirects may reach. This is your protection against server-side request forgery.

Change it when: the API has a path prefix (put it in the base URL), or one client must call several hosts.

Setting	Default	Notes
`builder(baseUrl)` (`baseUrl`)	required	Absolute `http` or `https` URL, optionally with a path (`https://api.example.com/v2`)
`allowAbsoluteUrls(boolean)`	`false`	Allow full URLs as request paths
`allowedHosts(Set<String>)`	the base URL's host	Hosts that requests and every redirect hop may reach

```
GroovyHttpClient.builder('https://api.example.com/v2')
        .allowAbsoluteUrls(true)
        .allowedHosts(['api.example.com', 'cdn.example.com'] as Set)
        .build()
```

17.2. Connection and timeouts

Controls: how long the client waits before giving up — on a connection, and on the server's answer.

Change it when: always consider it for production. Set the connect timeout short (the service is either reachable or not), and the request timeout to the service's slowest normal response plus a margin. Too long, and a slow dependency ties up your callers; too short, and you fail requests that would have succeeded.

`connectTimeout(Duration)`	10 s	TCP and TLS connection set-up
`requestTimeout(Duration)`	30 s	Until the response headers arrive. The body is then bounded by `idleTimeout` (per request) and the size limit.

A single slow request can override the request timeout: `timeout Duration.ofMinutes(2)` in its closure. The wait of a `*Sync` call is separate (30 s by default, or the `Duration` you pass).

17.3. Protocol

Controls: which HTTP version the client prefers. HTTP/2 multiplexes many requests over one connection; HTTP/3 runs over QUIC (UDP), which recovers better from packet loss on poor networks.

Change it when: a server misbehaves with HTTP/2 (force `HTTP_1_1`), or you want HTTP/3 where it is offered.

`httpVersion(v)`	`HTTP_2`	`HttpVersion.HTTP_1_1`, `HTTP_2` or `HTTP_3`; also `'HTTP/1.1'`, `'HTTP/2'`, `'HTTP/3'`
`http3()` / `http3(Http3Discovery)` (`http3Discovery`)	off	Prefer HTTP/3 with discovery `ANY` (default), `ALT_SVC` or `HTTP_3_URI_ONLY`; see Chapter 28

HTTP/2 is negotiated, so a server that only speaks HTTP/1.1 still works with the default.

17.4. TLS

Controls: which servers the client trusts, how it proves its own identity, and which TLS versions and ciphers it accepts.

Change it when: the service uses a private or self-signed certificate (trust store), requires a client certificate (key managers), or your security policy restricts protocols and ciphers.

`trustStore(KeyStore)`	JVM default (public CAs)	CAs or certificates to trust
`trustManagers` (key only)	none	Custom trust managers, instead of a trust store
`keyManagers(KeyManager…)`	none	Your client certificate, for mutual TLS
`sslContext(SSLContext)`	built for you	A complete context; overrides the three above
`allowedTlsProtocols(List)`	`TLSv1.3`, `TLSv1.2`	`production()` uses TLS 1.3 only
`allowedCipherSuites(List)`	JVM default	Restrict cipher suites
`enforceHostnameVerification(boolean)`	`true`	Check the certificate matches the host name
`insecureAllowed(boolean)`	`false`	Accept any certificate; only `trustAll()` sets it. Never in production.

```
def kmf = KeyManagerFactory.getInstance(KeyManagerFactory.defaultAlgorithm)
kmf.init(clientKeyStore, keyPassword)

GroovyHttpClient.builder('https://partner.example.com')
        .trustStore(partnerCaStore)
        .keyManagers(kmf.keyManagers)
        .allowedTlsProtocols(['TLSv1.3'])
        .build()
```

Section 15.2 shows how to create the stores; `SSL_CERTIFICATE_GUIDE.md` in the repository goes further.

17.5. Proxy

Controls: whether requests go direct or through an HTTP proxy, and with what credentials.

Change it when: your network requires a proxy the JVM is not already configured for, some hosts must bypass it, or you need to ignore the JVM's proxy settings.

`proxy(host, port)` (`proxyHost`, `proxyPort`)	JVM settings	An explicit HTTP proxy
`proxyAuth(user, password)` (`proxyUser`, `proxyPassword`)	none	Basic proxy credentials
`nonProxyHosts(String…)`	none	Hosts that bypass the proxy; `*` wildcards
`noProxy()`	off	Connect directly, ignoring JVM settings

See Chapter 24 for HTTPS tunnels and the JVM property Basic proxy authentication needs.

17.6. Redirects

Controls: whether 3xx responses are followed, and how many hops.

Change it when: you need the 3xx response itself (to read `Location` after creating something), or a service legitimately chains more than five redirects.

`allowRedirects(boolean)`	`true`	`false` returns the 3xx response to you
`maxRedirects(int)`	5	More hops raise `RedirectRejectedException`

Every hop is checked against the allowed hosts, HTTPS never redirects to HTTP, and credentials and cookies are removed when the origin changes. How the method and body change is described in Section 11.7.

17.7. Cookies

Controls: which cookies set by servers are kept and sent back.

Change it when: you want a stateless client (`ACCEPT_NONE`) or are testing against local servers (`ACCEPT_ALL`).

`cookiePolicy(CookiePolicy)`	`ACCEPT_ORIGINAL_SERVER`	See Chapter 12

17.8. Content: codecs and compression

Controls: how request values become bytes, how response bytes become values, and whether responses are compressed in transit.

Change it when: a service uses a media type the built-in codecs do not cover (a vendor JSON type is already covered by the `+json` suffix), you want Jackson with your own `ObjectMapper`, or the service serves gzip and you want to save bandwidth.

`codecs { … }` / `codecs(CodecRegistry)`	built-ins	Add or replace encoders and parsers; see [codecs]

`compression(boolean)`	`false`	Accept and decode gzip and deflate; see Chapter 26

```
GroovyHttpClient.builder(url)
        .codecs { parser 'application/x-yaml', { String text -> yaml.load(text) } }
        .compression(true)
        .build()
```

17.9. Authentication

Controls: the credentials sent with every request, and where they may go.

Change it when: the service needs credentials. Set them here rather than as a default header, so they get origin scoping, per-attempt refresh and redirect stripping.

```
GroovyHttpClient.builder(url).auth { bearer { -> tokens.current() } }.build()
client.auth { basic 'user', secret }              // replace later; null clears
```

See Chapter 23.

17.10. Default headers and response handlers

Controls: headers added to every request, and response handlers applied to every request.

Change it when: every request needs the same header (client identification, an API version), or the same outcome means the same thing everywhere (401 means the session expired).

```
client.withHeader('X-Client', 'orders-service')
GroovyHttpClient.builder(url).response { when(401) { r -> throw new SessionExpired() } }.build()
client.defaultResponse { when(503) { r -> null } }
```

See Chapter 8 and Section 20.1.

17.11. Resilience: retries and the circuit breaker

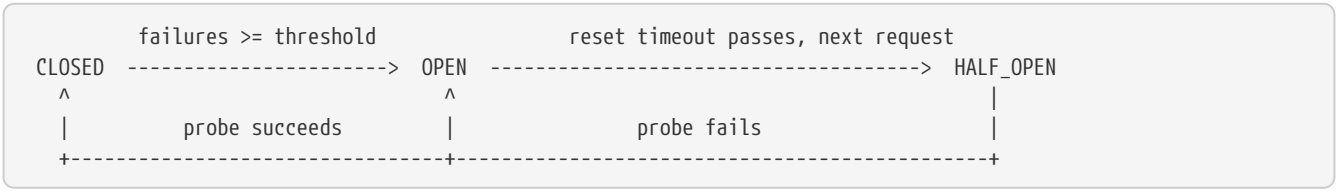
Controls: what happens when a call fails. Retries repeat a failed request after a pause, which rides out brief failures. The circuit breaker stops calling a service that keeps failing, so your application fails fast instead of piling up waiting requests, and the service gets room to recover.

Change it when: in production, always decide. Retry transient failures of idempotent requests; tune the breaker to how quickly you want to stop calling a broken service, and how often to check whether it is back.

`retryPolicy(RetryPolicy)`	none	Attempts, backoff, what to retry; see Chapter 27
`failureThreshold(int)`	5	Failures in a row that open the breaker
`resetTimeoutMs(long)`	30 000	How long the breaker stays open before letting a probe request through

The circuit breaker states

`CLOSED`	Normal: every request is sent. Network errors, timeouts and 5xx responses count as failures; any success resets the count. 4xx responses are answers and are not counted.
`OPEN`	Reached after `failureThreshold` failures in a row. Every request fails at once with `CircuitOpenException`, with no network call.
`HALF_OPEN`	Once `resetTimeoutMs` has passed since the last failure, the next request is let through as a probe; others are rejected while it runs. If it succeeds the breaker closes; if it fails, it opens again.



The move from `OPEN` to `HALF_OPEN` happens when a request arrives, so a client that is not being used stays `OPEN` until its next request. Inspect the state as described in Section 27.3.

Choosing values: a threshold of 3 to 10 avoids opening on one unlucky request; a reset timeout of 10 to 60 seconds gives a restarting service time to come up without leaving your application cut off for long. Retries happen inside the breaker, so a request with three attempts counts as one failure.

17.12. Limits

Controls: how much data the client accepts, protecting your memory from huge or endless responses.

Change it when: the service legitimately returns more than 10 MB (raise it, or better, stream), or long-lived event streams carry large items.

`maxResponseBytes(long)`	10 MB	Per body; decompressed bytes with compression, bytes written for downloads. `Long.MAX_VALUE` for none.
`maxStreamItemBytes(long)`	1 MiB	Per line, event or NDJSON value in a stream consumer
`errorBodyPreviewChars(int)`	0	Characters of the body in exception messages, 0 to 4096; keep 0 in production

17.13. Observability: logging and metrics

Controls: what the client tells you about its traffic — log lines per request, and counters you can publish.

Change it when: turn logging on to diagnose a problem or in non-production environments; keep metrics on (the default) and publish them to your monitoring.

`enableLogging(boolean)`	`false`	A credential-safe log line per request, retry and redirect; see Chapter 31

`enableMetrics(boolean)`	`true`	Per-client counters in `client.metrics`; see Section 27.5

```
def client = GroovyHttpClient.builder(url).enableLogging(true).build()
println client.metrics.snapshot()        // requests, outcomes, status classes, retries, latency...
```

17.14. Profiles

Controls: nothing by itself — a profile is a ready-made combination of the settings above. Start from the closest one and override what differs.

	`defaults`	`production`	`staging`	`testing`	`trustAll`
Certificates verified	yes	yes	yes	yes	**no**
TLS protocols	1.3, 1.2	1.3	1.3	1.3, 1.2	1.3, 1.2
Connect / request timeout	10 s / 30 s	5 s / 20 s	5 s / 30 s	10 s / 30 s	3 s / 30 s
Retries	none	3 attempts	3 attempts	none	2 attempts
Absolute URLs	no	no	no	yes	yes
Cookie policy	original server	original server	original server	accept all	accept all
Logging	off	off	on	on	on
Metrics	on	on	on	off	off
Error body in message	0	0	0	120 chars	0

```
new GroovyHttpClient(SecurityConfig.production(url).withOverrides(requestTimeout: Duration.ofSeconds(60)))
```

Chapter 18. Codecs and Data Transformation

HTTP moves bytes; your code works with maps, lists, strings and your own classes. **Codecs** translate between the two, chosen by media type, in both directions:

```
request:   your value  --encoder for the Content-Type-->  bytes on the wire
response:  bytes       --parser for the Content-Type---->  Map / List / GPathResult / String / your class
```

Every client has a `CodecRegistry`. It starts with the built-in codecs, picks up Jackson if it is on the classpath, and takes whatever you add.

18.1. Where codecs are used

`json(value)`, `form(map)`, `body(value, type)`	Encode the request body with the encoder for that type
`r.parsed`, `r.json`, `r.xml`	Parse the response body

`r.bodyAs(Type)`, `r as Type`, typed verbs (`getSync(Type, path)`)	Parse, then map onto your class
`e.json`, `e.parsed`, `e.bodyAs(Type)`, `e.problem` on `HttpResponseException`	The same for error bodies

18.2. The built-in codecs

Media types	Encodes (request)	Parses to (response)
`application/json`, `*/*+json`	Maps, lists and objects with Groovy's `JsonOutput`; a `String` or `byte[]` as given	`Map` / `List` / scalars (`JsonSlurper`); an empty body is `null`
`application/xml`, `text/xml`, `*/*+xml`	A `String`; a markup closure (`{ order(id: 7) { line(sku: 'A1') } }`, via `StreamingMarkupBuilder`); a `GPathResult`	`GPathResult` (`XmlSlurper`). DOCTYPE declarations and external entities are refused, so XXE and "billion laughs" attacks fail.
`text/csv`	A list of maps (header row = all keys, in first-seen order) or a list of lists (no header); RFC 4180 quoting, CRLF line ends	A list of maps, one per record, keyed by the header row; a record longer than the header fails
`application/x-www-form-urlencoded`	A map; list values repeat the key; `null` values skipped	`Map<String, List<String>>`
`text/plain`, `text/*`	`String.valueOf(value)`	`String`
`application/octet-stream`	`byte[]` only	`byte[]`
anything else (`*/*`)	no encoder: `body(value, type)` fails	`String` (or `byte[]` if you ask for `byte[]`)

Text is decoded and encoded with the `charset` of the media type, UTF-8 if none.

```
def bin = new GroovyHttpClient('https://httpbin.org')

bin.postSync('/post', null) { body([[sku: 'A1', qty: 2], [sku: 'B7', qty: 1]], 'text/csv') }
// sku,qty
// A1,2
// B7,1

bin.postSync('/post', null) {
    body({ order(id: 7) { line(sku: 'A1', qty: 2) } }, 'application/xml')
}
// <order id='7'><line sku='A1' qty='2'/></order>

def report = client.getSync('/reports/stock.csv').parsed      // List<Map>: [[sku:'A1', qty:'2'], ...]
```

18.3. Media type matching

The registry picks the most specific codec for the type:

1. the exact type — `application/vnd.acme.order+json` if you registered it;
2. the structured suffix — `+json` and `+xml` types fall to `*/*+json` and `*/*+xml`, so `application/problem+json`, `application/hal+json` and vendor types work with no set-up;

3. a type wildcard — `text/*`;
4. `*/*`.

Parameters other than `charset` (`; version=2`) do not affect the choice. Among codecs for the same pattern, yours win over a provider's (Jackson), which win over the built-ins.

18.4. Typed results: mapping onto your classes

`bodyAs(Type)` and typed verbs parse the body, then map it onto `Type`. Without Jackson the mapping is plain Groovy:

- A JSON object becomes an instance created with the no-argument constructor, its properties set from the object's members — recursively, for properties whose type is a class of yours.
- A JSON array becomes a `List` (elements as parsed), or an array of your class if you ask for `Order[]`.
- Numbers, strings and booleans are converted with Groovy's normal casting.
- **A member your class has no property for is an error** (`ContentParseException` naming the class and the member), so a renamed field is caught instead of silently lost. Jackson behaves the same by default.

```
class Line  { String sku; Integer qty }
class Order { Long id; String status; List<Line> lines; Customer customer }

Order o = client.getSync(Order, '/orders/42')
Order[] many = client.getSync(Order[], '/orders')
```

Classes need a no-argument constructor and settable properties (a normal Groovy class has both). For immutable classes, records, or properties named differently from the JSON, use Jackson. Elements of a `List<Line>` property stay maps without Jackson, because Groovy does not see the element type at run time; declare an array (`Line[] lines`) or use Jackson when you need them typed.

CSV and form bodies map the same way: each CSV record (a map) can become an instance of your class.

18.5. Jackson

When `jackson-databind` 2.x is on the classpath, the registry registers Jackson for `application/json` and `*/*+json`. It then handles typed results and the encoding of objects — so `@JsonProperty`, `@JsonIgnoreProperties`, records, Java time types and the rest of Jackson apply. Untyped access (`r.json`, `r.parsed`) stays on Groovy's parser, so the types you get back from it never change with the classpath.

To use your own configured `ObjectMapper`:

```
def mapper = new ObjectMapper().findAndRegisterModules()
        .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)

def client = GroovyHttpClient.builder(url).codecs { jackson mapper }.build()
```

18.6. Your own codecs

Register a parser, an encoder, or both, for any media type pattern:

```
def client = GroovyHttpClient.builder(url).codecs {
    // YAML in and out, with SnakeYAML
    parser  'application/x-yaml', { String text -> new Yaml().load(text) }
    encoder 'application/x-yaml', { body -> new Yaml().dump(body) }

    // a vendor JSON type with its own mapper
    parser 'application/vnd.tmf+json', { String text, MediaType type, Class target -> tmfMapper.readValue(text,
target) }

    // protocol buffers, as bytes
    parser  'application/x-protobuf', { byte[] bytes, MediaType type, Class target -> target.parseFrom(bytes) }
    encoder 'application/x-protobuf', { msg -> msg.toByteArray() }
}.build()

def r = client.postSync('/config', null) { body([replicas: 3], 'application/x-yaml') }
Map cfg = client.getSync('/config').parsed
```

- A **parser** closure whose first parameter is a `String` receives the decoded text; otherwise it receives the raw `byte[]`. It may also take the `MediaType` and the requested `Class`, and should return a value of that class when one is asked for.
- An **encoder** closure receives the body value (and optionally the `MediaType`) and returns `byte[]` or a `CharSequence` (encoded with the type's charset).
- For reuse across projects, implement `org.softwood.http.codec.CodecProvider` and list the class in `META-INF/services/org.softwood.http.codec.CodecProvider`: every client then picks it up automatically, as it does Jackson. `codecs { withoutProviders() }` switches provider discovery off for one client.

To share one set of codecs between several clients, build a registry once:

```
CodecRegistry shared = CodecRegistry.builder()
        .parser('application/x-yaml', { String t -> new Yaml().load(t) })
        .build()

def a = GroovyHttpClient.builder(urlA).codecs(shared).build()
def b = GroovyHttpClient.builder(urlB).codecs(shared).build()
```

18.7. Transforming data in the request and response

Codecs handle the format; everything else is ordinary Groovy around the call. A few common shapes:

```
// project a response: only the fields you need
def summaries = client.getSync('/orders').json.collect { [id: it.id, total: it.total] }

// reshape before sending: rename keys to the API's names
client.postSync('/customers', null) { json(customer.properties.subMap(['name', 'email']) + [externalRef:
customer.id]) }

// convert between formats: CSV in, JSON out
def rows = files.getSync('/exports/stock.csv').parsed
api.postSync('/stock', null) { json(rows.collect { [sku: it.sku, qty: it.qty as int] }) }

// typed result through a response handler, with a default
Order o = client.getSync(Order, '/orders/42') { response { when(404) { new Order(id: 42, status: 'unknown') } } }
```

Chapter 19. Request Configuration

Everything you can put in a request closure, grouped by purpose. Request settings describe **this** request: what to send, and anything that differs from the client's defaults. Where a request setting has a client counterpart, the request's value wins for that request only.

19.1. URI and query

Controls: which resource, and which subset of it. **Use it for** identifiers in the path, and filters, sorting and paging in the query (Chapter 10).

`query(Map)`	Add query parameters, URL-encoded; a list value repeats the parameter

The path is the verb's first argument. A path may carry its own query string, and with `allowAbsoluteUrls` it may be a full URL.

19.2. Headers

Controls: request metadata. **Use it for** content negotiation (`Accept`), conditional requests (`If-None-Match`), tracing ids and per-request API versions. Not for credentials: use `auth { }`.

`header(name, value)`	Add a value (default headers are kept)
`setHeader(name, value)`	Replace every value of the header, default headers included

19.3. Cookies

Controls: cookies for this request only; they are not stored. **Use it for** values that vary per request. Session cookies are handled for you (Chapter 12).

`cookie(name, value)`	Send a cookie with this request
`cookies(Map)`	Several at once

19.4. Content

Controls: the request body and its `Content-Type`. **Use** `json` for most APIs, `form` for form endpoints, `body(value, type)` for other formats, and the streaming forms for large uploads (Chapter 11).

`json(value)`	JSON body; maps, lists and objects are serialised, strings sent as is
`form(Map)`	URL-encoded form body
`body(value, contentType)`	Encoded by the codec for the type (CSV, XML, your own…)
`body(value)`	Type from the value: `byte[]` octet-stream, text plain, anything else JSON
`body(Path)` / `body(File)`	Stream a file (Section 25.5)
`body(InputStream, type)` / `body(Flow.Publisher, type)`	Stream a one-shot body

`gzipBody()`	Compress the body as it is sent (Chapter 26)

19.5. Multipart

Controls: `multipart/form-data` uploads of files and fields together.

Multipart bodies are built with `postMultipart` / `postMultipartSync` (Section 11.6). With a list of parts, the closure is a normal request closure.

19.6. Authentication

Controls: credentials for this request. **Use it for** a call that needs different credentials (an admin token) or none at all (a public health check).

`auth { … }`	Credentials for this request, replacing the client's (Chapter 23)
`noAuth()`	No credentials for this request

19.7. Compression

Controls: whether this response may arrive compressed, and whether this request body is sent compressed. **Use it for** one large transfer on a client that otherwise does not compress, or the reverse.

`compression(boolean)`	Accept compressed responses for this request, overriding the client
`gzipBody()`	Send the body gzip-compressed

19.8. Timeouts

Controls: how long this request may take. **Use it for** the one slow endpoint (a report, an export) on a client whose timeout suits everything else, and `idleTimeout` for streams that should fail when they go quiet.

`timeout(Duration)`	Request timeout for this request (until the response headers)
`idleTimeout(Duration)`	Fail if the body delivers no bytes for this long

19.9. Retry

Controls: the retry policy for this request, replacing the client's. **Use it for** an idempotent call you want retried harder, or a call you must not retry at all (`maxAttempts 1`).

```
client.getSync('/flaky') {
    retry {
        maxAttempts 4
        backoffExponential(200, 2_000, true)
        on5xx(true)
    }
}
```

The options are the same as for a client retry policy (Chapter 27).

19.10. Protocol

Controls: the HTTP version for this request. Rarely needed: set it on the client.

`httpVersion(HttpVersion)`	Prefer a version for this request
`http3()` / `http3(Http3Discovery)`	Use HTTP/3 for this request
`version(HttpClient.Version)`	The same with the JDK enum
`expectContinue(boolean)`	Send `Expect: 100-continue`

19.11. Response handling

Controls: what this call returns for each outcome, and how much it may read. **Use it for** outcomes that are normal answers here (a 404 on a lookup), and the occasional large response.

`response { ⋯ }`	Handlers for this request (Section 20.1)
`maxResponseBytes(long)`	Size limit for this request

19.12. Streaming targets

Controls: where the response body goes instead of memory. **Use it for** any verb whose answer is a file or a stream — typically a POST that returns an export or an event stream.

`downloadTo(Path / File / OutputStream)`	Write a 2xx body there instead of memory
`lines { line → }`	Deliver the body line by line
`sse { event → }`	Deliver Server-Sent Events
`ndjson { value → }`	Deliver parsed NDJSON values

A request has at most one streaming target. See Chapter 25.

Chapter 20. Response Configuration

20.1. Status handlers

```
Order order = client.getSync(Order, '/orders/42') {
    response {
        success   { HttpClientResponse r -> r.bodyAs(Order) }    // status below 400
        when(404) { r -> null }
        when(409, 422) { r -> throw new ConflictException(r.problem) }
        when(500..599) { r -> cachedOrder() }
        failure   { r -> throw r.toException() }                 // any other status 400+
    }
}
```

The most specific handler wins: `when(int)`, then `when(a, b⋯)` or `when([list])`, then `when(range)`, then `success` or `failure`. A status no handler matches behaves as without handlers: returned below 400, raised at 400 and above.

Handlers run once, on the final outcome, after retries and the circuit breaker. A handler's return value is the call's result; a handler that throws fails the call with that exception.

Getting the value. The untyped verbs return `HttpClientResponse`. To receive what a handler returns, use a typed verb: `getSync(Order, path)` checks the value is an `Order`, and `getSync(Object, path)` accepts anything. If a handler returns the response itself, a typed verb converts it with `bodyAs`.

Client-wide handlers. Set them on the builder with `response { }` or later with `client.defaultResponse { }`. A request's own `response { }` is merged over them, key by key.

20.2. Exception handler

```
response {
    exception { Throwable t -> log.warn("orders unavailable: ${t.class.simpleName}"); cachedOrders() }
}
```

`exception` receives the real cause of a failure that is not a status code — `ConnectException`, `HttpTimeoutException`, `CircuitOpenException`, `ResponseTooLargeException`, `RedirectRejectedException` — never a `CompletionException` wrapper.

20.3. Parsers and typed results

`r.parsed`	Parsed by `Content-Type` with the client's codecs
`r.json`, `r.xml`	JSON (Map/List) and XML (GPathResult)
`r.bodyAs(Type)` / `r as Type`	Mapped onto your class
`getSync(Type, path)`	A typed verb

Add parsers for your own media types with `codecs { parser … }` (Chapter 18).

20.4. Headers

`r.getHeader(name)`	First value, case-insensitive
`r.headers`	All headers, `Map<String, List<String>>`
`r.contentType`, `r.charset`	From `Content-Type`
`r.httpVersion`	The protocol actually used

20.5. Decoded content

With compression on, a gzip or deflate body is decoded before you see it. `r.decoded` is then `true`, `r.contentEncoding` names what the server applied, and the `Content-Encoding` and `Content-Length` headers are removed because they no longer describe the body. See Chapter 26.

20.6. Size limits

A body larger than `maxResponseBytes` stops with `ResponseTooLargeException` (status 413, with `limit` and

`bytesRead`). Raise the limit for one request with `maxResponseBytes n` in its closure, or stream the body to a file.

20.7. Stream consumers

`lines`, `sse` and `ndjson` deliver the body as it arrives instead of returning it; `downloadTo` writes it to a file or stream. See Chapter 25.

Chapter 21. Verb Forms

The **verbs** are the client methods that send requests, named after the HTTP method they send: `get`, `post`, `put`, `patch`, `delete`, `head`, `options`. Each comes in a small, regular set of forms, so once you know the pattern you know every method.

21.1. Anatomy of a call

```
Order order = client.postSync(Order, '/orders', null) { json(newOrder) }
//                   |   |     |      |         |     |
//                   |   |     |      |         |     +-- request closure: the DSL (optional)
//                   |   |     |      |         +-------- body string (post, put, patch only; often null)
//                   |   |     |      +------------------ path, appended to the base URL
//                   |   |     +-------------------------- result type (optional): a typed result
//                   |   +-------------------------------- Sync: wait for the result (omit for a future)
//                  +------------------------------------ the HTTP method
```

Three choices make up every form:

1. **Sync or async** — `getSync` waits and returns the result; `get` returns a `CompletableFuture` immediately.
2. **Untyped or typed** — without a type you get an `HttpClientResponse`; with `get(Type, …)` you get a `Type`.
3. **With or without a closure** — the closure configures the request; leave it out when the defaults suffice.

21.2. The forms

Form	Returns	Use it when
`getSync(path)`	`HttpClientResponse`	A plain request, result needed now
`getSync(path) { … }`	`HttpClientResponse`	The request needs query, headers, a body or settings
`getSync(path, closure, Duration)`	`HttpClientResponse`	You need a different wait than 30 s
`getSync(Type, path) { … }`	`Type`	You want your own class back, or a response handler's value
`get(path) { … }`	`CompletableFuture<HttpClientResponse>`	Concurrency or composition
`get(Type, path) { … }`	`CompletableFuture<Type>`	The same, typed

Form	Returns	Use it when
`postSync(path, String body) { … }`	`HttpClientResponse`	`post`, `put`, `patch`: a body string, or `null` and build it in the closure
`postSync(Type, path, body) { … }`	`Type`	Typed POST, PUT, PATCH

`head`, `options` and `delete` take the same arguments as `get`; `delete` also has typed forms. `head` and `options` have no typed forms, since their answers are headers.

```
def r      = client.getSync('/orders/42')                                  // untyped, sync
Order o    = client.getSync(Order, '/orders/42')                           // typed, sync
def future = client.get('/orders/42') { header 'X-Trace', traceId }        // untyped, async
CompletableFuture<Order> f = client.get(Order, '/orders/42')               // typed, async
Order made = client.postSync(Order, '/orders', null) { json(newOrder) }    // typed POST
client.deleteSync('/orders/42')
def h      = client.headSync('/files/report.pdf')
```

21.3. Typed forms in detail

The type argument does two jobs:

- **Without response handlers**, a 2xx body is mapped onto the type ([codecs]); a status of 400 or more still throws.
- **With response handlers**, the handler's return value is the result. It must be an instance of the type (or `null`); a handler that returns the response itself has it mapped onto the type.

```
Order o   = client.getSync(Order, '/orders/42')                              // mapped
Order m   = client.getSync(Order, '/orders/42') { response { when(404) { null } } }
Object v  = client.getSync(Object, '/flags') { response { success { r -> r.json.enabled } } }
String s  = client.getSync(String, '/readme.txt')                            // the body as text
byte[] b  = client.getSync(byte[], '/logo.png')                              // the raw bytes
```

Use `Object` as the type when handlers return different types for different outcomes.

21.4. Streaming verbs

Streaming verbs deliver the body somewhere other than memory (Chapter 25):

Form	Returns	Use it when
`downloadSync(path, Path / File / OutputStream) { … }`	`StreamResult`	A large body to a file or stream
`download(…)`	`CompletableFuture<StreamResult>`	The same, without waiting
`linesSync(path) { String line → }`	`StreamResult`	Text that arrives line by line
`sseSync(path) { SseEvent e → }`	`StreamResult`	Server-Sent Events
`ndjsonSync(path) { Map value → }`	`StreamResult`	Newline-delimited JSON
`lines`, `sse`, `ndjson`	`CompletableFuture<StreamResult>`	The same, without waiting

Form	Returns	Use it when
`downloadBytesSync(path)` / `downloadBytes(path) { … }`	`byte[]`	A small binary body straight into an array

For `lines`, `sse` and `ndjson`, the **last** closure is your consumer; to configure the request as well, pass a request closure before it:

```
client.sseSync('/events', { header 'Last-Event-ID', lastId; idleTimeout Duration.ofSeconds(30) }) { SseEvent e ->
    handle(e)
}
```

A consumer may take a second `StreamControl` parameter to stop early: `{ String line, StreamControl c → if (done) c.stop() }`.

Any verb can stream its response by putting a streaming target in the request closure — for a POST that answers with a file or an event stream:

```
StreamResult r = client.postSync(StreamResult, '/exports', null) { json(query); downloadTo Path.of('export.csv')
}
client.postSync(StreamResult, '/v1/chat', null) { json(prompt); sse { SseEvent e -> print e.data } }
```

21.5. Multipart verbs

`postMultipartSync(path) { b → b.part { … } }`	`HttpClientResponse`	Parts described in the multipart DSL
`postMultipartSync(path, List<MultipartPart>) { … }`	`HttpClientResponse`	Parts built in code; the closure configures the request
`postMultipart(…)`	`CompletableFuture<HttpClientResponse>`	The same, without waiting

21.6. Writing the calls in Groovy

- A closure as the last argument goes outside the parentheses: `client.getSync('/x') { … }`. With no other arguments the parentheses can go too: `client.getSync '/x'`.
- `null` for the body of `post`, `put` and `patch` is normal: the closure sets the body.
- A request closure can be stored and reused: `def asJson = { header 'Accept', 'application/json' }` then `client.getSync('/x', asJson)`.

21.7. Exceptions

The `*Sync` forms throw the request's exception directly (`HttpResponseException`, `HttpTimeoutException`…). The asynchronous forms complete the future exceptionally with the same exception; `join()` wraps it in `CompletionException`, `get()` in `ExecutionException`, so in asynchronous code prefer handling failures in the chain or with a `response { exception { … } }` handler.

Chapter 22. Error Handling

22.1. HttpResponseException

A status of 400 or above, after retries, becomes `HttpResponseException`. Its message is a fixed, log-safe summary:

```
HTTP 404 Not Found (GET https://api.example.com/orders/42, 1284 bytes, application/problem+json)
```

The reason phrase comes from the RFC 9110 table (HTTP/2 and HTTP/3 send none); the URI has no user info and no query string; and there is no body. Bodies routinely echo request data and tokens, and messages end up in logs. To include the start of the body, set `errorBodyPreviewChars` (0 to 4096; the `testing()` profile uses 120).

The detail is on the exception:

Field	Type	Notes
`statusCode`	`int`	
`reasonPhrase`	`String`	
`method`	`String`	
`uri`	`URI`	The full URI, query string included: log it with care
`responseHeaders`, `header(name)`	`Map` / `String`	Case-insensitive
`contentType`	`MediaType`	Null if the response had none
`responseBody`	`String`	Decoded with the response charset
`responseBytes`	`byte[]`	A copy
`json`, `parsed`, `bodyAs(Type)`	`Object`	Parsed with the client's codecs
`problem`	`ProblemDetail`	RFC 9457 detail, or null
`response`	`HttpClientResponse`	The response, when available

The exception is serialisable; after deserialisation the parsing accessors throw `IllegalStateException` (the codecs are not serialised) while the raw fields still work.

22.2. Problem details

When the error body is `application/problem+json`, `e.problem` gives `type`, `title`, `status`, `detail`, `instance` and `extensions` (any other members). `r.problem` does the same inside a response handler. `application/problem+xml` is not parsed.

22.3. ResponseTooLargeException

Raised when a body exceeds `maxResponseBytes`. It extends `HttpResponseException` and keeps status 413, so existing `catch` blocks still see it, and adds `limit` and `bytesRead`. It is raised the same way on every protocol, and it is not retried.

22.4. All exceptions

Exception	When	Retried?
`HttpResponseException`	Status 400 or above	By status, per policy
`ResponseTooLargeException`	Body over `maxResponseBytes`	No
`java.net.http.HttpTimeoutException`	No headers within the request timeout, or a body idle past `idleTimeout`	Per policy
`IOException` (`ConnectException`…)	Connection refused or reset, TLS failure	Per policy
`CircuitOpenException`	The circuit breaker is open	No
`RedirectRejectedException`	A redirect broke policy, or would re-send a one-shot body	No
`ContentParseException`	A body could not be parsed or mapped onto a class	No
`ContentDecodingException`	A gzip/deflate body is corrupt or truncated	No
`StreamItemTooLargeException`	A line or event exceeded `maxStreamItemBytes`	No
`AuthenticationException`	An OAuth2 token request failed, or a proxy refused the tunnel	No
`IllegalArgumentException`	A host that is not allowed, or an invalid setting	No
`java.util.concurrent.TimeoutException`	Your own `*Sync` wait expired	—

The `*Sync` methods throw these directly; asynchronous calls complete their future exceptionally with them.

Advanced Features

Chapter 23. Authentication

23.1. The schemes

```
def bin = GroovyHttpClient.builder('https://httpbin.org').auth { basic 'user', 'passwd' }.build()
assert bin.getSync('/basic-auth/user/passwd').json.authenticated

auth { basic 'user', 'password' }                        // RFC 7617, UTF-8
auth { bearer 'eyJhbGciOi...' }                           // a fixed token
auth { bearer { -> vault.read('orders-token') } }         // a supplier, asked on every attempt
auth { apiKey header: 'X-API-Key', value: key }           // in a header
auth { apiKey query: 'api_key', value: key }              // in the query string
auth {                                                    // OAuth2 client credentials
    oauth2 {
        tokenUrl 'https://login.example.com/oauth2/token'
        clientId 'orders-service'
        clientSecret secret
        scope 'orders.read', 'orders.write'
    }
}
```

Declare credentials on the builder (`.auth { }`), on a client (`client.auth { }`; `null` clears), or on one request (`auth { }` in its closure, or `noAuth()` to send none). An `auth { }` block holds one scheme. Credentials are applied on each attempt, so retries pick up a refreshed token; if a request already has the header (or query parameter), yours wins.

23.2. Where credentials go

Credentials are sent only to the base URL's origin — same scheme, host and port. A request to another host, even an allowed one, gets none unless you name it:

```
auth {
    bearer token
    authHosts 'files.example.com'
}
```

On a redirect to another origin, `Authorization`, `Proxy-Authorization`, `Cookie` and a custom API-key header are removed. Secrets never appear in `toString()`, exception messages or logs.

23.3. OAuth2 client credentials

- The token is fetched on first use and cached until `expires_in` minus 60 seconds. Concurrent requests share one token request.
- A 401 invalidates the token; the client fetches a new one and re-sends the request once, if its body can be re-sent.
- The client authenticates with `client_secret_basic`; use `clientAuth 'post'` for `client_secret_post`.

- Other options: `skew Duration.ofSeconds(120)` to refresh earlier, `param 'audience', '…'` for extra form parameters.
- The token endpoint must be HTTPS, except on a loopback host or when the client allows insecure connections (for local test servers).
- The token request uses the client's TLS settings, trust store and proxy, but not its allowed hosts, retry policy or interceptors — you do not need to add the identity provider to `allowedHosts`.

A failed token request raises `AuthenticationException` with the token endpoint's status and its OAuth `error` code, never the body, the secret or a token:

```
OAuth2 token request to https://login.example.com/oauth2/token failed: HTTP 401 (invalid_client)
```

Digest authentication is not supported. For other schemes, set the `Authorization` header in an interceptor (Chapter 29).

Chapter 24. Proxies

Without proxy settings, the client uses the JVM's default proxy selector, which follows the standard properties (`https.proxyHost`, `http.nonProxyHosts`…). To set a proxy explicitly:

```
def client = GroovyHttpClient.builder('https://api.example.com')
        .proxy('proxy.corp.example', 3128)
        .proxyAuth('svc-user', proxyPassword)          // optional
        .nonProxyHosts('*.internal', 'localhost')      // optional
        .build()

GroovyHttpClient.builder(url).noProxy().build()        // direct, ignoring the JVM settings
```

For an `https://` target the client tunnels through the proxy with `CONNECT`, and TLS runs end to end with the target. The allowed-hosts check and the redirect policy apply to the target, never to the proxy.

24.1. Proxy authentication

`proxyAuth` sends Basic credentials in `Proxy-Authorization`, up front, on every request the proxy carries — never on a request that bypasses it.



For `https://` targets the JDK removes Basic proxy credentials from `CONNECT` unless the JVM is started with

```
-Djdk.http.auth.tunneling.disabledSchemes=""
```

It is a JVM-wide security setting, so the library leaves it to you. Without it the proxy answers 407 and the request fails with an `AuthenticationException` that says so. Plain `http://` targets are not affected.

24.2. SOCKS and HTTP/3

- **SOCKS is not supported.** The JDK `HttpClient` ignores SOCKS proxies and connects directly, so the

client refuses a SOCKS proxy when it is built, and logs a warning (once) if the JVM's own settings select one.

- **HTTP/3 cannot go through an HTTP proxy** (QUIC is UDP). With a proxy, HTTP/3-only clients are refused and other HTTP/3 settings fall back to HTTP/2.

Chapter 25. Streaming in Depth

25.1. Downloads

```
StreamResult r = client.downloadSync('/exports/2026-10.csv', Path.of('exports/2026-10.csv'))
r.statusCode; r.bytesRead; r.path; r.contentType; r.headers

client.download('/big.iso', Path.of('big.iso'))           // CompletableFuture<StreamResult>
client.downloadSync('/big.iso', new File('big.iso'))
client.downloadSync('/report.pdf', outputStream)          // your stream: flushed, never closed
client.downloadSync('/archive.zip', target) { maxResponseBytes 5_000_000_000L }
```

- The body goes to a temporary file next to the target and is moved into place only when complete. A failure deletes the temporary file and leaves an existing target untouched.
- Only a final 2xx body is written. Error bodies are kept in memory for `HttpResponseException`; the target is never created.
- `maxResponseBytes` counts the bytes written (decoded bytes with compression).
- A download to a file may be retried by the retry policy; one to an `OutputStream` only until its first byte is written.
- `downloadTo(…)` in any request closure streams that request's body, so a POST that answers with a file works too. Use the typed verb to get the `StreamResult`: `client.postSync(StreamResult, '/exports', json) { downloadTo target }`.

25.2. Lines, Server-Sent Events and NDJSON

```
client.linesSync('/logs/tail') { String line -> println line }
client.ndjsonSync('/feed') { Map record -> store(record) }
client.sseSync('/events') { SseEvent e -> println "${e.event}: ${e.data}" }

client.sseSync('/events', { header 'Last-Event-ID', lastSeen }) { SseEvent e -> handle(e) }   // with request
settings

// any verb: e.g. an API that answers a POST with an event stream
client.postSync(StreamResult, '/v1/chat', JsonOutput.toJson(prompt)) {
    header 'Content-Type', 'application/json'
    sse { SseEvent e -> print e.data }
}
```

- Items arrive in order, one at a time; the next bytes are read only after your consumer returns.
- A consumer that throws fails the request with that exception. A stream is never retried once an item has been delivered.
- An error status raises `HttpResponseException`; the consumer never runs.

- Lines end at `\n`, `\r\n` or `\r`, decoded with the `Content-Type` charset (UTF-8 by default; UTF-16 is not supported).
- `SseEvent` has `event` (`message` if unnamed), `data` (multi-line data joined with `\n`), `id` (persists across events) and `retry`. The client does not reconnect; send the last `id` as `Last-Event-ID` to resume.
- The `StreamResult` reports `items`, `bytesRead` and `stopped`.

25.3. Stopping early

```
client.linesSync('/logs/tail') { String line, StreamControl c ->
    if (line.contains('READY')) c.stop()
}
```

After `stop()` nothing more is delivered, the response is cancelled, and the call completes normally with `stopped == true`, on every protocol. Cancelling the future also ends the stream.

25.4. Limits and timeouts for long streams

- `maxStreamItemBytes` (1 MiB by default) bounds any single line, event or value; over it, `StreamItemTooLargeException`.
- `maxResponseBytes` still bounds the whole body, so raise it for a stream meant to run for hours: `{ maxResponseBytes Long.MAX_VALUE }`.
- The request timeout ends at the response headers. Use `idleTimeout Duration.ofSeconds(30)` to fail a stream that goes silent; time spent in your consumer does not count.
- The stream `*Sync` methods have no wait limit of their own.

25.5. Uploads

```
client.putSync('/files/42', null) { body Path.of('big.bin') }             // Content-Length set; replayable
client.putSync('/files/42', null) { body new File('data.csv'), 'text/csv' }
client.postSync('/ingest', null) { body inputStream, 'application/x-ndjson' }      // chunked; one-shot
client.postSync('/ingest', null) { body publisher, 'application/octet-stream' }    // Flow.Publisher; one-shot
```

A file's content type is the one you give, else probed from its name, else `application/octet-stream`; a missing file fails before anything is sent. A file body is re-sent on retry and on 307/308 redirects. An `InputStream` or publisher body is sent once: it is never retried, and a 307/308 redirect is refused.



A hot publisher such as `SubmissionPublisher` drops items emitted before anyone subscribes. Start emitting once `numberOfSubscribers > 0`.

Chapter 26. Compression

The JDK client does not decompress responses; this library can, opt-in:

```
def client = GroovyHttpClient.builder('https://httpbin.org').compression(true).build()

def r = client.getSync('/gzip')
assert r.json.gzipped          // decoded for you
```

```
assert r.decoded
assert r.contentEncoding == 'gzip'

client.getSync('/gzip') { compression false }     // per request, either way
```

- The client sends `Accept-Encoding: gzip, deflate` and decodes exactly those (gzip including multi-member, deflate in both the zlib and raw forms). Other encodings are passed through untouched. If you set `Accept-Encoding` yourself, nothing is decoded.
- **The size limit counts decompressed bytes.** Decoding happens inside the bounded reader, so a small body that would inflate to gigabytes stops at the limit. This holds for buffered responses, error bodies, downloads and streams.
- A decoded response has no `Content-Encoding` or `Content-Length` header (they described the compressed bytes); `r.contentEncoding` keeps what the server applied.
- A corrupt or truncated body raises `ContentDecodingException`; it is not retried.

26.1. Compressing requests

```
client.postSync('/post', null) {
    body(records, 'application/json')
    gzipBody()                        // Content-Encoding: gzip
}
```

`gzipBody()` works for every kind of body, including multipart and streamed files. It is never automatic — many servers refuse compressed requests — so use it only where the server accepts them.

Chapter 27. Retries, Circuit Breaker and Metrics

27.1. Retries

A retry policy repeats a request that failed for a reason likely to pass: a dropped connection, a timeout, a 502 from a restarting service.

```
import org.softwood.http.GroovyHttpClient.RetryPolicy

def policy = new RetryPolicy.Config().tap {
    maxAttempts 3                          // including the first attempt
    backoffExponential(200, 2_000, true)   // initial ms, max ms, jitter
    onIoExceptions(true)                   // connection refused, reset...
    onTimeouts(true)
    on5xx(true)                            // or onStatusCodes([502, 503, 504])
}.build()

def client = GroovyHttpClient.builder(url).retryPolicy(policy).build()

client.getSync('/flaky') { retry { maxAttempts 5; on5xx(true) } }    // per request
```

`maxAttempts(n)`	Total attempts, at least 1
`backoffExponential(initialMs, maxMs, jitter)`	Delay doubling from `initialMs` up to `maxMs`; with `jitter` each delay is randomised, so many clients do not retry in lockstep

`onIoExceptions(boolean)`, `onTimeouts(boolean)`	Retry network failures and timeouts
`on5xx(boolean)`, `onStatusCodes(codes)`	Retry these statuses (429 and 503 are common additions)
`retryNonIdempotent(boolean)`	Also retry POST and PATCH (off by default)

Never retried, whatever the policy: POST and PATCH (unless `retryNonIdempotent`), one-shot uploads, streams that have delivered an item, downloads to an `OutputStream` that have written a byte, and failures that would only repeat — size limits, parse and decoding errors, policy refusals.



Only enable `retryNonIdempotent` when the server de-duplicates retried requests, for example with an `Idempotency-Key` header.

Choosing a policy. Two or three attempts with jittered backoff covers most transient failures. More attempts mostly add load to a service that is already struggling — that case is the circuit breaker's job.

27.2. The circuit breaker

Retries help with failures that last milliseconds. When a service is down for minutes, retrying every request makes things worse: your callers wait, threads and connections pile up, and the struggling service gets hammered as it tries to recover. The circuit breaker detects this and fails fast instead.

Each client has one breaker, covering every request it makes. It has three states (Section 17.11.1):

- **CLOSED** — normal. Counted failures (network errors, timeouts, 5xx) build up; any success resets the count.
- **OPEN** — after `failureThreshold` failures in a row (default 5). Requests fail immediately with `CircuitOpenException`; nothing is sent.
- **HALF_OPEN** — once `resetTimeoutMs` (default 30 s) has passed since the last failure, the next request is let through as a probe. Success closes the breaker; failure opens it again for another `resetTimeoutMs`.

```
def client = GroovyHttpClient.builder(url).failureThreshold(3).resetTimeoutMs(10_000).build()

try {
    client.getSync('/orders')
} catch (GroovyHttpClient.CircuitOpenException e) {
    return cachedOrders()             // degrade gracefully while the service recovers
}
```

Or as a value, with a response handler:

```
List orders = client.getSync(List, '/orders') {
    response {
        success { r -> r.json }
        exception { Throwable t -> t instanceof GroovyHttpClient.CircuitOpenException ? cachedOrders() : null }
    }
}
```

4xx responses are not failures for the breaker: they are answers. Retries happen inside the breaker, so one request with three attempts counts once.

27.3. Inspecting the breaker

```
import org.softwood.http.GroovyHttpClient.State

client.circuitState                    // State.CLOSED, OPEN or HALF_OPEN

def s = client.circuitBreakerStatus    // a read-only snapshot
s.state                                // State.OPEN
s.failureCount                         // 3
s.failureThreshold                     // 3
s.resetTimeout                         // PT10S
s.openedAt                             // Instant, or null when not open
s.lastFailureAt                        // Instant, or null
s.open                                 // true
s.probeDue                             // true once the reset timeout has passed: the next request is the probe
```

Because `OPEN` becomes `HALF_OPEN` only when a request arrives, an idle client reads `OPEN` after its reset timeout has passed; `probeDue` tells you the next request will be let through.

Health checks. Report the breaker in your application's health endpoint, so an open circuit is visible to operators and load balancers:

```
Map health() {
    def s = ordersClient.circuitBreakerStatus
    [orders: [status: s.open && !s.probeDue ? 'DOWN' : 'UP', circuit: s.state.name(), failures: s.failureCount]]
}
```

27.4. Reacting to state changes

Register a listener to be told of every transition — to log it, raise an alert, or record a metric:

```
client.onCircuitStateChange { State from, State to ->
    log.warn("orders circuit ${from} -> ${to}")
    if (to == State.OPEN) alerts.raise('orders service unavailable')
    if (to == State.CLOSED) alerts.clear('orders service unavailable')
}
```

Listeners run on the thread that caused the change, so keep them quick. A listener that throws is logged and ignored; several listeners may be added.

27.5. Collecting metrics

Every client counts its traffic (unless built with `enableMetrics(false)`; the `testing()` profile turns it off):

```
def m = client.metrics.snapshot()

m.requests              // verb calls, including those the breaker rejected
m.succeeded             // completed with status < 400
m.failed                // ended in an exception: status 400+, network error, timeout, circuit open
m.status2xx             // also status1xx, status3xx, status4xx, status5xx
m.attempts              // requests sent, retries included
m.retries
m.timeouts
m.circuitRejections
m.failuresByType        // e.g. [HttpResponseException: 4, ConnectException: 1]
m.averageLatency        // Duration, call to outcome
```

```
m.maxLatency

println m.toMap()       // all of it as a flat map, for logging
client.metrics.reset()  // zero every counter
```

A request is counted once, whatever its retries, when its outcome is known — before response handlers run, so a 404 that a handler turns into `null` still counts as a 4xx failure. Counters are `LongAdder`s; leaving metrics on costs next to nothing.

Publishing to Micrometer

Register gauges that read the counters, and a listener for the breaker state:

```
import io.micrometer.core.instrument.Gauge
import io.micrometer.core.instrument.MeterRegistry

void bindMetrics(GroovyHttpClient client, String name, MeterRegistry registry) {
    def m = client.metrics
    Map<String, Closure<Number>> counters = [
            'requests'          : { m.snapshot().requests },
            'failed'            : { m.snapshot().failed },
            'retries'           : { m.snapshot().retries },
            'circuit.rejections': { m.snapshot().circuitRejections },
            'latency.max.ms'    : { m.snapshot().maxLatency.toMillis() }
    ]
    counters.each { metric, read ->
        Gauge.builder("http.client.${metric}", m, { read() as double }).tag('client', name).register(registry)
    }
    Gauge.builder('http.client.circuit.open', client, { it.circuitState == State.OPEN ? 1d : 0d })
         .tag('client', name).register(registry)
}
```

For per-request timings with percentiles (rather than totals), use an interceptor to record each attempt into a Micrometer `Timer` (Chapter 29).

Logging a summary

Without a metrics system, log a snapshot periodically and reset, so each line covers one interval:

```
scheduler.scheduleAtFixedRate({
    log.info("orders client: ${client.metrics.snapshot().toMap()}")
    client.metrics.reset()
}, 1, 1, TimeUnit.MINUTES)
```

Chapter 28. HTTP/3 in Depth

HTTP/3 runs HTTP over QUIC, on UDP. It needs JDK 26 or later (JEP 517), TLS 1.3, and a server that offers it.

```
def client = GroovyHttpClient.builder('https://cloudflare-quic.com').http3().build()
println client.getSync('/').httpVersion        // HTTP_2: the server announces HTTP/3 in Alt-Svc
println client.getSync('/').httpVersion        // HTTP_3 from now on

client.getSync('/') { http3() }                // or for one request
GroovyHttpClient.isHttp3Available()            // true on JDK 26+
```

28.1. Discovery modes

`Http3Discovery.ANY` (default)	Use HTTP/3 when the JDK knows the server offers it — some servers from the first request, others once their `Alt-Svc` header has been seen — and HTTP/2 otherwise
`Http3Discovery.ALT_SVC`	Start on HTTP/2 and switch when the server advertises HTTP/3 in `Alt-Svc`
`Http3Discovery.HTTP_3_URI_ONLY`	HTTP/3 or fail — no fallback

28.2. Fallback

With `ANY` or `ALT_SVC`, the same code works everywhere:

- On a JDK without HTTP/3, the client logs a warning when it is created and uses HTTP/2 (`HTTP_3_URI_ONLY` fails instead).
- A server that does not offer HTTP/3, or a network that blocks UDP 443 (common behind corporate firewalls), gives HTTP/2.
- Through an HTTP proxy, HTTP/3 is not possible; the client uses HTTP/2 (Chapter 24).

Check `r.httpVersion` to see what was used. To test the network path itself, try `HTTP_3_URI_ONLY` against one of the public endpoints in Chapter 1.

Chapter 29. Interceptors

An interceptor wraps every request attempt, for logging, metrics, tracing or short-circuiting:

```
client.addInterceptor { ctx, next ->
    long started = System.nanoTime()
    next.call(ctx).thenApply { resp ->
        metrics.timer(ctx.method).record(System.nanoTime() - started)
        resp
    }
}
```

- `ctx` carries the request's `method`, `uri` and `attempt` number.
- Call `next.call(ctx)` to continue; it returns a `CompletableFuture` of the response. Return your own completed future to answer without a network call (a cache, for example).
- Interceptors run in the order added: the first added is outermost, so it sees the request first and the response last.
- They run inside the retry loop, once per attempt, and receive 4xx and 5xx responses as responses, not exceptions.
- An exception thrown by an interceptor fails the request with that exception.
- Credentials from `auth { }` are added after the interceptors, so an interceptor never sees or logs them.

29.1. Per-attempt timings with Micrometer

The client's own metrics (Section 27.5) are totals per client. For latency percentiles per method, time

each attempt in an interceptor:

```
client.addInterceptor { ctx, next ->
    def sample = Timer.start(registry)
    next.call(ctx).whenComplete { resp, err ->
        sample.stop(registry.timer('http.client.attempts',
                'method', ctx.method, 'status', resp ? String.valueOf(resp.statusCode) : 'error'))
    }
}
```

Chapter 30. URL Builder

`org.softwood.url.URLBuilder` builds URLs with encoded query parameters, for when you need a URL string rather than a request — a link in an email, an absolute URL for `allowAbsoluteUrls` clients, a value for a test.

```
import org.softwood.url.URLBuilder

String url = new URLBuilder('api.example.com')          // the argument is the host
        .port(8443)
        .path('/v1/orders')
        .param('status', 'open')
        .param('tag', ['a', 'b'])                        // repeated: tag=a&tag=b
        .build()
// https://api.example.com:8443/v1/orders?status=open&tag=a&tag=b

String same = new URLBuilder({
    protocol 'https'
    host 'api.example.com'
    path 'v1/orders'                                     // a leading '/' is added
    params(status: 'open', page: 2)
}).build()
```

- The protocol defaults to `https`; the host is required (`build()` throws `IllegalStateException` without one).
- Parameter names and values are encoded as form data, so a space becomes `+`.
- `isValidURL(String)` checks a string parses as a URI.

Logging and Testing

Chapter 31. Logging

The library logs through SLF4J and ships no logging backend; add the binding your application uses (`logback-classic`, `log4j-slf4j2-impl`…).

Request logging is off by default. Turn it on per client:

```
def client = GroovyHttpClient.builder(url).enableLogging(true).build()
```

At `INFO`, logger `org.softwood.http.GroovyHttpClient`, you then get one line per completed attempt, retry and redirect:

```
[GroovyHttpClient] GET https://api.example.com/orders -> 200 in 84 ms (attempt 1, HTTP_2)
[GroovyHttpClient] retrying GET https://api.example.com/orders in 412 ms (attempt 2 of 3) after status 503
[GroovyHttpClient] following 303 redirect https://api.example.com/orders -> https://api.example.com/orders/42
```

These lines are credential-safe: URIs are logged without user info or query string, and headers and bodies are never logged.

Whether or not request logging is on, the client logs a `WARN` when:

- certificate verification is disabled (`trustAll()` / `insecureAllowed`);
- HTTP/3 was requested but is unavailable (old JDK, or a proxy), so HTTP/2 is used;
- the JVM's proxy settings select a SOCKS proxy, which the JDK client ignores;
- the circuit breaker rejects a request.

logback.xml

```
<logger name="org.softwood.http" level="INFO"/>
```

For counts and latencies rather than individual lines, use the client's metrics (Section 27.5); for more detail per request — headers, timings, payload sizes — add an interceptor (Chapter 29) and log what you need, taking care not to log credentials.

To log circuit breaker transitions, register a listener: `client.onCircuitStateChange { from, to → log.warn("circuit ${from} → ${to}") }` (Section 27.4).

Chapter 32. Testing Your Code

The library's own testing tools are published as **test fixtures**, so your tests can run against a real local HTTP server instead of mocks of the client. They are incubating: the API may still change before 3.0. This chapter is the reference; Chapter 7 is a guided tour with worked examples.

32.1. Adding the fixtures

Gradle

```
testImplementation testFixtures('io.github.woodmawa:groovy-httpclient:2.1.0')
```

Maven

```
<dependency>
    <groupId>io.github.woodmawa</groupId>
    <artifactId>groovy-httpclient</artifactId>
    <version>2.1.0</version>
    <classifier>test-fixtures</classifier>
    <scope>test</scope>
</dependency>
```

`MockHttpServer`	A real HTTP server with canned responses per method and path, and a record of what it received. Works with any test framework.
`HttpClientTestDsl`	A fluent layer: `when('/path').respondWith { … }` and `verify('/path').wasCalled()…`
`HttpClientTestTrait`	Brings the DSL into a Spock specification, with server lifecycle handled for you

All three serve plain HTTP on a free local port; create the client with `SecurityConfig.testing(url)`, which allows `http://localhost` and puts the start of error bodies into exception messages.

32.2. Asynchronous tests

Wait for futures with Groovy 6's `await` (it rethrows the original exception), and for side effects — requests reaching the server, callbacks, listeners — with Awaitility, written as `Awaitility.await()` because Groovy 6's own `await` shadows the static import. Worked examples are in Section 7.5.

32.3. MockHttpServer

```
import org.softwood.test.MockHttpServer

class OrderClientSpec extends Specification {
    MockHttpServer server = new MockHttpServer()
    def setup()   { server.init() }
    def cleanup() { server.shutdown() }

    def "creates an order"() {
        given:
        def check = server.addRequestCheck('POST', '/orders', 201)
                .withResponseBody('{"id":42}')
                .withResponseHeaders(['Content-Type': 'application/json'])
        def client = new GroovyHttpClient(SecurityConfig.testing("http://localhost:${server.port}"))

        when:
        def r = client.postSync('/orders', null) { json([sku: 'A1']) }

        then:
        r.json.id == 42
        check.lastRequest.header('Content-Type').startsWith('application/json')
        check.lastRequest.bodyText == '{"sku":"A1"}'
```

```
        check.received.size() == 1
    }
}
```

`addRequestCheck(method, path, status)`	Respond to `method path` with `status`
`withResponseBody(String)` / `withResponseBytes(byte[])`	The body (bytes for binary or gzip content)
`withResponseHeaders(Map)` / `withResponseCookies(Map)`	Response headers and cookies
`withExpectedHeaders(Map)` / `withExpectedCookies(Map)`	Answer 400 unless the request carries these
`withDelay(ms)`	Wait before answering (timeouts, slow servers)
`received`, `lastRequest`	What arrived: `method`, `uri`, `rawQuery`, `query`, `form`, `header(name)`, `body`, `bodyText`

32.4. HttpClientTestDsl

```
import org.softwood.http.HttpClientTestDsl

def server = HttpClientTestDsl.create().start()

server.when('/users/1').respondWithJson([id: 1, name: 'Alice'])
server.when('/users/2').respondWithStatus(404)
server.when('/search').respondWith {
    status 200
    contentType 'text/plain'
    body 'found'
}

def client = server.createClient()          // a client for the server's base URL
client.getSync('/users/1')

server.verify('/users/1').wasCalled(1).withMethod('GET').withHeader('Accept')
server.verify('/users/3').wasNotCalled()
server.recordedRequests.size()              // everything received, in order

server.close()
```

`respondWith` closures can set `status`, `contentType`, `header(name, value)` and the body with `body`, `json`, `xml`, `text` or `html`. Verifications can also check `withQueryParam`, `withBody`, `withBodyContaining` and `withJsonBody { json → … }`; `first()` and `last()` return recorded requests.

32.5. HttpClientTestTrait

```
import org.softwood.http.HttpClientTestTrait

class UserServiceSpec extends Specification implements HttpClientTestTrait {

    def "fetches a user"() {
        given:
        testServer.when('/users/1').respondWithJson([id: 1, name: 'Alice'])
        def client = createTestClient()

        expect:
        client.getSync('/users/1').json.name == 'Alice'
```

```
        testServer.verify('/users/1').wasCalled()
    }
}
```

The trait starts one server, lazily, shared by all features of the specification, and provides:

`testServer`, `testServerUrl`	The `HttpClientTestDsl` server and its base URL
`createTestClient()`	A client using `SecurityConfig.testing(…)`; `createTestClient { requestTimeout = Duration.ofSeconds(1) }` overrides settings
`mockEndpoint(path) { … }`	Shorthand for `testServer.when(path).respondWith { … }`
`verifyRequest(path)`, `recordedRequests`	Shorthands for verification and the recorded requests
`resetTestServer()`, `stopTestServer()`	Clear endpoints and recordings; stop the server

Call `resetTestServer()` in `setup()` when features must not see each other's endpoints and recordings.

Reference

Chapter 33. Examples

Complete scripts. Each runs as it is with `groovy script.groovy` (Groovy 6, Java 25+): the `@Grab` line fetches the library. They call public test services, so results vary a little from run to run.

33.1. Synchronous

Read JSON from a REST API

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

api.getSync('/users').json.each { u ->
    println "${u.id}  ${u.name.padRight(25)} ${u.email}"
}
```

Typed results, and "not found" as a value

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

class Todo { Integer userId; Integer id; String title; Boolean completed }

def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

Todo todo = api.getSync(Todo, '/todos/1')
println "${todo.title} (done: ${todo.completed})"

Todo missing = api.getSync(Todo, '/todos/0') {
    response { when(404) { r -> null } }
}
println "missing: ${missing}"
```

Create, update and delete a resource

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def api = new GroovyHttpClient('https://api.restful-api.dev')

def created = api.postSync('/objects', null) {
    json([name: 'Groovy HTTP Client test', data: [year: 2026, price: 1.0]])
}.json
println "created ${created.id}"

def updated = api.putSync("/objects/${created.id}", null) {
    json([name: 'Groovy HTTP Client test (updated)', data: [year: 2026, price: 2.0]])
}.json
println "updated to ${updated.data.price}"

api.deleteSync("/objects/${created.id}")
println 'deleted'
```

Submit a form, follow the redirect, keep the cookie

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def bin = new GroovyHttpClient('https://httpbin.org')

def r = bin.postSync('/redirect-to?url=/get&status_code=303', null) { form(user: 'alice') }
println "ended at ${r.json.url} with ${r.statusCode}"          // the 303 was followed with a GET

bin.getSync('/cookies/set?session=abc123')
println "server sees cookies: ${bin.getSync('/cookies').json.cookies}"
```

Upload a file with multipart

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient
import org.softwood.http.MultipartPart

def bin = new GroovyHttpClient('https://httpbin.org')
def csv = 'sku,qty\nA1,2\nB7,1\n'.bytes

def r = bin.postMultipartSync('/post', [
        MultipartPart.text('description', 'stock take'),
        MultipartPart.bytes('file', 'stock.csv', 'text/csv', csv)
])
println r.json.form.description       // a text part is a form field
println r.json.files.file             // a part with a filename is a file
```

Authenticate, and handle errors

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient
import org.softwood.http.GroovyHttpClient.HttpResponseException

def ok = GroovyHttpClient.builder('https://httpbin.org').auth { basic 'user', 'passwd' }.build()
println ok.getSync('/basic-auth/user/passwd').json

def wrong = GroovyHttpClient.builder('https://httpbin.org').auth { basic 'user', 'nope' }.build()
try {
    wrong.getSync('/basic-auth/user/passwd')
} catch (HttpResponseException e) {
    println e.message            // HTTP 401 Unauthorized (GET https://httpbin.org/basic-auth/user/passwd, 0
bytes)
}
```

33.2. Asynchronous

Fan out and collect

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

long start = System.currentTimeMillis()
def futures = (1..20).collect { id -> api.get("/posts/${id}") }
def titles = futures*.join().collect { it.json.title }
println "${titles.size()} posts in ${System.currentTimeMillis() - start} ms"
```

Compose dependent requests

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def api = new GroovyHttpClient('https://jsonplaceholder.typicode.com')

api.get('/posts/1')
   .thenCompose { post -> api.get("/users/${post.json.userId}") }
   .thenAccept { user -> println "post 1 was written by ${user.json.name}" }
   .join()
```

A deadline with a fallback

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient
import java.time.Duration

def bin = new GroovyHttpClient('https://httpbin.org')

def result = bin.get(Object, '/delay/5') {
    timeout Duration.ofSeconds(2)
    response {
        success   { r -> 'fresh' }
        exception { Throwable t -> "fallback (${t.class.simpleName})" }
    }
}.join()
println result       // fallback (HttpTimeoutException)
```

33.3. Streaming

Download a file

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient
import java.nio.file.Path

def bin = new GroovyHttpClient('https://httpbin.org')
def r = bin.downloadSync('/image/png', Path.of('pig.png'))
println "${r.bytesRead} bytes -> ${r.path}"
```

Read an NDJSON stream

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def bin = new GroovyHttpClient('https://httpbin.org')
def r = bin.ndjsonSync('/stream/5') { Map line -> println "record ${line.id}" }
println "${r.items} records"
```

Follow live Server-Sent Events, then stop

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import groovy.json.JsonSlurper
import org.softwood.http.GroovyHttpClient
import org.softwood.http.SseEvent
import org.softwood.http.StreamControl

def wiki = new GroovyHttpClient('https://stream.wikimedia.org')
def json = new JsonSlurper()
```

```
// Wikimedia refuses requests without a descriptive User-Agent (403)
wiki.sseSync('/v2/stream/recentchange', {
    header 'User-Agent', 'my-app/1.0 (me@example.com)'
    maxResponseBytes Long.MAX_VALUE
}) { SseEvent e, StreamControl c ->
    def edit = json.parseText(e.data)
    println "${edit.wiki}: ${edit.title}"
    if (c.items == 10) c.stop()
}
```

Compressed responses

```
@Grab('io.github.woodmawa:groovy-httpclient:2.1.0')
import org.softwood.http.GroovyHttpClient

def bin = GroovyHttpClient.builder('https://httpbin.org').compression(true).build()
def r = bin.getSync('/gzip')
println "decoded ${r.decoded}, server sent ${r.contentEncoding}, gzipped: ${r.json.gzipped}"
```

Chapter 34. Upgrading

34.1. From 2.0 to 2.1

2.1 is source-compatible with 2.0, and every new capability is opt-in. Check five behaviour changes:

Change	Who notices	What to do
`HttpResponseException.message` has no response body	Code or alerts matching on message text	Read `responseBody`, `bodyAs(…)` or `problem`; or set `errorBodyPreviewChars`
The client's own size limit raises `ResponseTooLargeException`	Code checking `statusCode == 413` for that case	Catch the subclass (status is still 413)
Query values are encoded once (`&`, `=`, `/`, `?`, non-ASCII arrive intact)	Servers or tests that worked around double encoding	Remove the workaround
`addCookie` sends RFC 6265 cookies (no `$Version`)	Servers expecting RFC 2965 attributes	None expected
Multipart text and JSON parts are plain form fields: `MultipartPart.text/json` and DSL `content` parts no longer get a generated filename (`name.txt`)	Servers that read those parts as file uploads	Read them as fields, or give the part a `filename`

Three 2.0 bugs are fixed: a name of yours (a constant, say) used inside a request closure could send a stray `GET /<name>`; an oversized body on HTTP/2 or HTTP/3 surfaced as a retryable `IOException`; and `@CompileStatic` code could not use the request DSL. `MockHttpServer.withDelay` is now honoured. If you copied the test helpers into your project, depend on the published fixtures instead (Section 32.1).

New settings (`compression`, `maxStreamItemBytes`, `errorBodyPreviewChars`, the proxy keys) need 2.1, because `SecurityConfig` refuses unknown names.

`enableMetrics`, accepted but unused in 2.0, now switches the per-client counters in `client.metrics` (on by

default, off in `testing()` and `trustAll()`). The circuit breaker can now be inspected (`circuitState`, `circuitBreakerStatus`) and observed (`onCircuitStateChange`). The `import_certificate.sh` SSL tool, which in 2.0 contained the Windows batch script by mistake, now works on Linux and Mac.

34.2. From 1.x to 2.0

2.0 requires Java 25 and Groovy 6 and changed several security-relevant defaults:

TLS	`new GroovyHttpClient(url)` verifies certificates (1.x accepted any). `testing()` verifies too; only `trustAll()` does not. `trustStore`, `trustManagers` and `keyManagers` take effect (1.x ignored them).
Settings	Unknown `SecurityConfig` names are refused; explicit `false` and `0` are honoured (1.x treated them as "use the default").
Redirects	Followed by the library, each hop checked against `allowedHosts`, never HTTPS to HTTP, credentials removed across origins.
Exceptions	`*Sync` methods throw the real exception, not `ExecutionException`.
Retries	5xx retries work; POST and PATCH are retried only with `retryNonIdempotent(true)`.
Interceptors	Receive 4xx and 5xx as responses, not exceptions.
Request DSL	Per-request settings such as `timeout(…)` apply to every attempt; an unsupported call fails instead of being ignored.
Validation	Cookie values and multipart headers are checked for injection.
Circuit breaker	Half-open admits a single probe.
Logging	No bundled backend: add your own SLF4J binding.
Lifecycle	`close()` releases the underlying `HttpClient`.

Chapter 35. Troubleshooting

Common symptoms, their usual cause, and the fix. If yours is not here, turn on `enableLogging(true)` (Chapter 31) and check the exception's fields (Section 22.1).

Symptom	Cause	Fix
`javax.net.ssl.SSLHandshakeException: PKIX path building failed`	The server's certificate is not trusted by the JVM trust store. Common after upgrading from 1.x, where the String constructor and `testing()` trusted every certificate.	Trust the issuing CA: `builder(url).trustStore(caStore)`, `withOverrides(trustStore: caStore)`, or an `sslContext`. Use `trustAll()` only for throwaway local debugging.
`SSLHandshakeException` / connection reset against an mTLS endpoint	The server requires a client certificate and none was sent.	Supply `keyManagers` (builder `.keyManagers(kms)` or `withOverrides(keyManagers: kms)`).

Symptom	Cause	Fix
`IllegalArgumentException: Unknown SecurityConfig setting(s): [⋯]`	A misspelt or unsupported key passed to `withOverrides` or the constructor.	Use one of the names listed in the message. 1.x ignored such keys silently.
`GroovyHttpClient.RedirectRejectedException`	A redirect pointed to a host outside `allowedHosts`, from HTTPS to HTTP, to a non-HTTP scheme, or exceeded `maxRedirects`.	Add the legitimate target host to `allowedHosts`, or raise `maxRedirects`. Never "fix" an HTTPS → HTTP redirect on the client.
A 3xx response instead of the final resource	`allowRedirects(false)` is set (it is honoured since 2.0.0).	Remove the setting if you want redirects followed.
`ResponseTooLargeException` (status 413) that the server never sent	The response body exceeded `maxResponseBytes` (decompressed bytes when compression is on); the client stopped reading.	Raise the limit for that request with `maxResponseBytes n` in the request closure, or stream the body to a file with `download(⋯)` (Chapter 25).
`HttpResponseException` message no longer shows the response body	Since 2.1 the message is log-safe and carries no body.	Read `e.responseBody`, `e.bodyAs(Type)` or `e.problem`, or set `errorBodyPreviewChars` (Section 22.1).
`IllegalStateException: A response handler returned ⋯`	A `response { }` handler returned a value from an untyped verb, which returns `HttpClientResponse`.	Use the typed verb, e.g. `getSync(Order, path)` or `getSync(Object, path)` (Section 20.1).
`ContentParseException` naming a class and a key	Typed mapping without Jackson found a JSON key with no matching property.	Add the property, or put Jackson on the classpath and configure it to ignore unknown properties (Chapter 18).
`ContentDecodingException`	A body labelled `gzip` or `deflate` is truncated, corrupt or not compressed at all.	A server or intermediary problem; turn `compression` off for that endpoint while it is fixed.
Response body is binary gibberish with `Content-Encoding: gzip`	Compression is off, or you set `Accept-Encoding` yourself, so the client did not decode it.	Turn on `compression(true)` and leave `Accept-Encoding` to the client (Chapter 26).
`StreamItemTooLargeException`	One line, event or NDJSON value exceeded `maxStreamItemBytes` (default 1 MiB).	Raise `maxStreamItemBytes` if the items are legitimately that large.
A long-lived SSE or line stream ends with `ResponseTooLargeException`	The whole stream reached `maxResponseBytes`.	Raise it for that request: `{ maxResponseBytes Long.MAX_VALUE }` (Section 25.2).
`RedirectRejectedException: A 307 redirect ⋯ one-shot body`	An `InputStream` or `Publisher` upload was redirected with 307/308, which would need the body sent twice.	Upload from a `Path` or `byte[]`, or upload to the final URL directly (Section 25.5).
An upload from a `SubmissionPublisher` arrives empty	A hot publisher drops items emitted before the client subscribes.	Start emitting once `numberOfSubscribers > 0`, or use a cold publisher.

Symptom	Cause	Fix
`AuthenticationException: OAuth2 token request … failed: HTTP 401 (invalid_client)`	The identity provider rejected the client id or secret, or the client authentication method.	Check the credentials; try `clientAuth 'post'` if the provider wants `client_secret_post` (Section 23.3).
`AuthenticationException: OAuth2 tokenUrl must be HTTPS`	The token endpoint is plain HTTP on a non-loopback host.	Use the provider's HTTPS endpoint.
No `Authorization` header on a request to another host	Credentials go only to the base URL's origin.	Add the host with `authHosts(…)` inside `auth { }` (Chapter 23).
`AuthenticationException: The proxy refused the HTTPS tunnel (407 …)`	The proxy wants credentials: none were set, they are wrong, or the JDK dropped Basic credentials from `CONNECT`.	Set `proxyAuth`, and start the JVM with `-Djdk.http.auth.tunneling.disabledSchemes=""` (Section 24.1).
`IllegalArgumentException: SOCKS proxies are not supported`	A SOCKS proxy was configured; the JDK `HttpClient` would silently connect directly.	Use an HTTP proxy (Section 24.2).
WARN: `The JVM proxy settings select a SOCKS proxy … Requests are going direct`	`socksProxyHost` (or system proxy settings) pick a SOCKS proxy, which the JDK `HttpClient` ignores.	Configure an HTTP proxy with `proxy(host, port)`, or `noProxy()` if direct is intended.
`testFixtures("io.github.woodmawa:groovy-httpclient:…")` does not resolve	Using 2.0 (no fixtures published) or a Maven build.	Use 2.1 or later; in Maven use `<classifier>test-fixtures</classifier>` (Section 32.1).
`IllegalArgumentException: Absolute URLs disabled by configuration` / `Target host not allowed`	An absolute URL was passed while `allowAbsoluteUrls` is off, or its host isn't in `allowedHosts`.	Use a path relative to the base URL, or enable `allowAbsoluteUrls` and list the host.
`IllegalArgumentException: Invalid value for cookie …`	The cookie value contains `;`, `,`, whitespace, quotes, backslash or control characters.	URL-encode or Base64 the value.
`IllegalArgumentException: MultipartPart.filename must not contain CR, LF or NUL`	A part name, filename or content type carries line breaks, often from unsanitised user input.	Sanitise the value; take only the file name component of user-supplied paths.
`MissingMethodException` inside a request closure	A call that the request DSL doesn't support. 1.x silently ignored it.	See the request DSL table in Chapter 19.
`GroovyHttpClient.CircuitOpenException`	The breaker is OPEN, or HALF_OPEN with its single probe already in flight.	Back off or serve a fallback; tune `failureThreshold` / `resetTimeoutMs`.
A POST is not retried despite a retry policy	Since 2.0.0, non-idempotent methods are retried only with `retryNonIdempotent(true)`.	Opt in only when the server de-duplicates (e.g. an `Idempotency-Key`).
No log output with `enableLogging(true)`	No SLF4J binding on the classpath (2.0.0 no longer ships logback), or the `org.softwood.http.GroovyHttpClient` logger is above INFO.	Add a binding such as `logback-classic` and configure the logger level.

Symptom	Cause	Fix
`IllegalStateException: HTTP/3-only … requires JDK 26`	`Http3Discovery.HTTP_3_URI_ONLY` on a JDK without HTTP/3.	Run on JDK 26+, or use `ANY` / `ALT_SVC`, which fall back to HTTP/2.
`response.httpVersion` is `HTTP_2` although `http3()` is set	JDK < 26 (see the WARN at client creation), the server doesn't offer HTTP/3, or UDP 443 is blocked (common behind corporate firewalls).	Expected fallback with `ANY` / `ALT_SVC`. To check the path, try `HTTP_3_URI_ONLY` against the server.
`No Awaitable adapter found for type …` in your own Groovy 6 code	A method of yours named `await(…)` is shadowed by Groovy 6's built-in `await`.	Rename the method.

Chapter 36. API Map and Licence

36.1. Classes you use

`org.softwood.http.GroovyHttpClient`	The client and its verbs. Nested: `Builder`, `GroovyRequestBuilder` (the request DSL), `HttpClientResponse`, `RetryPolicy`, and the exceptions `HttpResponseException`, `ResponseTooLargeException`, `CircuitOpenException`, `RedirectRejectedException`, `ContentParseException`
`SecurityConfig`	All client settings and the profiles
`ResponseHandlers`	The `response { }` DSL
`ProblemDetail`	RFC 9457 problem details
`StreamResult`, `StreamControl`, `SseEvent`	Streaming results and control
`ClientMetrics` (and `ClientMetrics.Snapshot`)	Per-client counters: `client.metrics`
`CircuitBreakerStatus`, `GroovyHttpClient.State`	Circuit breaker inspection: `client.circuitBreakerStatus`, `client.circuitState`
`MultipartPart`	Multipart parts
`HttpVersion`, `Http3Discovery`	Protocol preferences
`ContentDecodingException`, `StreamItemTooLargeException`, `AuthenticationException`	Other exceptions (Section 22.4)
`org.softwood.http.codec.CodecRegistry`, `MediaType`, `Encoder`, `Parser`, `CodecProvider`	Codecs
`org.softwood.http.auth.AuthPolicy`, `OAuth2ClientCredentials`	The `auth { }` DSL and its schemes
`org.softwood.url.URLBuilder`	URL building

`org.softwood.test.MockHttpServer`, `org.softwood.http.HttpClientTestDsl`, `HttpClientTestTrait`	Test fixtures (incubating)

Classes in `org.softwood.http.compression`, `.streaming` and `.proxy` are internal and may change in any release.

The complete API is in the Groovydoc, published as the `-javadoc` artifact; run `./gradlew groovydoc` to build it locally.

36.2. Getting help

- Source and issues: https://github.com/woodmawa/groovy-HttpClient
- Email: will.woodman.softwood@outlook.com

36.3. Licence

Copyright 2025 Softwood Consulting Ltd. Licensed under the Apache License, Version 2.0 (https://www.apache.org/licenses/LICENSE-2.0). Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.